Artificial Intelligence

Exam Revision Notes — Chapters 1 to 8

Based on George F. Luger, *Artificial Intelligence: Structures and Strategies for Complex Problem Solving*, 6th edition

1 AI: History and Applications
2 The Predicate Calculus
3 Structures and Strategies for State Space Search
4 Heuristic Search
5 Stochastic Methods
6 Building Control Algorithms for State Space Search
7 Knowledge Representation
8 Strong Method Problem Solving

How to use: each chapter has definitions in blue boxes, algorithms in grey boxes, and an **Exam focus** box in red listing what to be able to do. Textbook figures are reproduced from the book for study use. A one-page cheat sheet is at the end.

Contents

Chapter 1 — AI: History and Applications 4

1.1 Defining AI . . . 4
1.2 Historical foundations . . . 5
1.3 The Turing test (1950) . . . 5
1.4 Agents, situated and social models of intelligence . . . 6
1.5 Application areas of AI . . . 6
1.6 Summary: common features of AI (Luger's 8 points) . . . 7

Chapter 2 — The Predicate Calculus 8

2.1 Propositional calculus . . . 8
2.2 Predicate calculus: syntax . . . 8
2.3 Semantics of predicate calculus . . . 9
2.4 The blocks world . . . 10
2.5 Inference rules . . . 11
2.6 Unification . . . 11
2.7 The financial advisor (logic version) . . . 12

Chapter 3 — Structures and Strategies for State Space Search 13

3.1 Representation and search (Part II introduction) . . . 13
3.2 Graph theory . . . 13
3.3 Finite state machines and state space definition . . . 14
3.4 Data-driven versus goal-driven search . . . 16
3.5 Implementing graph search . . . 16
3.6 State space for logic: and/or graphs . . . 19

Chapter 4 — Heuristic Search 22

4.1 Heuristics . . . 22
4.2 Hill climbing and dynamic programming . . . 22
4.3 Best-first search . . . 24
4.4 Admissibility, monotonicity, informedness . . . 26
4.5 Heuristics in two-person games: minimax . . . 27
4.6 Complexity . . . 28

Chapter 5 — Stochastic Methods 30

5.1 Counting . . . 30
5.2 Elements of probability . . . 30
5.3 Bayes' theorem . . . 32
5.4 Applications . . . 32

Chapter 6 — Building Control Algorithms for State Space Search 34

6.1 Recursion-based search . . . 34

6.2 Production systems . . . 34
6.3 The blackboard architecture . . . 38

Chapter 7 — Knowledge Representation **39**

7.1 Three approaches to AI representation . . . 39
7.2 Associationist theories and semantic networks . . . 39
7.3 Scripts and frames . . . 41
7.4 Conceptual graphs (Sowa) . . . 43
7.5 Alternative representations and ontologies . . . 44
7.6 Agent-based and distributed problem solving . . . 45
7.7 Issues in representation (epilogue) . . . 45

Chapter 8 — Strong Method Problem Solving **47**

8.1 Expert system overview . . . 47
8.2 Rule-based expert systems . . . 48
8.3 Model-based, case-based and hybrid systems . . . 50
8.4 Planning . . . 51

Cheat sheet: key formulas, algorithms and comparisons **54**

Chapter 1 — AI: History and Applications

Purpose of the chapter: give **background** (philosophy, mathematics, psychology), an attempt at a **definition** of AI, and an **overview of application areas**.

1.1 Defining AI

> **Luger's definition**
>
> - AI is the branch of computer science concerned with the **automation of intelligent behaviour**. It is part of computer science, so it rests on sound theoretical and applied principles: **data structures** (knowledge representation), **algorithms** (applying that knowledge), and **languages and programming techniques** (implementation).

- **Weakness:** intelligence itself is not well defined. We recognise intelligent behaviour when we see it, but cannot specify it well enough to evaluate a program.
- **Open questions:** Is intelligence one faculty or a collection? How much is learned versus a priori? What is learning, creativity, intuition, self-awareness? Can it be inferred from behaviour alone, or does it need a particular internal mechanism? Must we copy human intelligence, or is an engineering approach enough? Can intelligence exist on a computer without biological experience?
- **AI as a test-bed:** theories of intelligence can be written as programs and tested by running them.
- **Working definition:** AI is the collection of problems and methodologies studied by AI researchers. AI is a human endeavour and is best understood in that context.

1.2 Historical foundations

Person / event	Contribution
Prometheus, Frankenstein	Myth of knowledge as transgression (nous); Shelley's *Modern Prometheus* shows nature can be analysed and understood systematically.
Aristotle	**Matter vs form** (basis for symbolic computing and data abstraction). **Logic** as the "instrument" (organon). The **syllogism**: all men are mortal, Socrates is a man, so Socrates is mortal (modus ponens).
Copernicus, Galileo	Split between mind and world: ideas about things are distinct from things themselves. Mathematics as a tool to describe the world.
Francis Bacon (1620)	*Novum Organum*: algorithm for finding the essence of an entity from positive instances and near-miss negative instances. Echoed in **Version Space Search** and the BACON discovery program.
Napier, Schickard, Pascal, Leibniz	Logarithms and Napier's bones (1614); Calculating Clock (1623); Pascaline (1642, add/subtract with carries); Leibniz wheel. Showed human arithmetic can be automated.
Descartes	**Mind–body dualism** (res cogitans vs res extensa). Structure of ideas need not equal structure of their subject matter, so mental processes can be studied in their own right.
Rationalists	Plato, Descartes, Pascal, Leibniz: reason and mathematics reconstruct the world. Early planners (STRIPS) and the Physical Symbol System Hypothesis have this flavour.
Empiricists	Aristotle (De Anima), Hobbes, Locke, Hume: "nothing enters the mind except through the senses". **Association** is the mechanism. Hume: impressions vs ideas; scepticism about causality. Influences semantic networks, MOPs, connectionism.
Bayes (1763)	Addressed Hume's causality questions mathematically; basis of Bayesian belief networks.
Kant	Synthesis: *a priori* (subject's reason) plus *a posteriori* (experience). Influences machine learning and constructivist epistemology.
Leibniz, Euler	Calculus philosophicus (first formal logic system). Euler: graph theory (Königsberg), basis of **state space search**.
Babbage, Lovelace	Difference Engine; Analytical Engine (store and mill = memory and processor, digital, punched cards). Lovelace: the engine "weaves algebraical patterns" (abstraction of form).
Boole (1847, 1854)	AND (*), OR (+), NOT (¬). Law **X*X = X**, hence values restricted to {0, 1}. Basis of binary arithmetic and logic circuits.
Frege	First-order predicate calculus: predicates, functions, quantified variables. Foundation of AI representation.
Whitehead and Russell	*Principia Mathematica*: mathematics as a purely formal system (strings and syntactic rules), so reasoning can be mechanised.
Tarski	Theory of reference: precise link from formulae to the world. Basis of formal semantics.
Digital computer (1940s)	Provided memory and processing power. Intelligence viewed as **information processing**; search owes more to sequential computers than to biology.

1.3 The Turing test (1950)

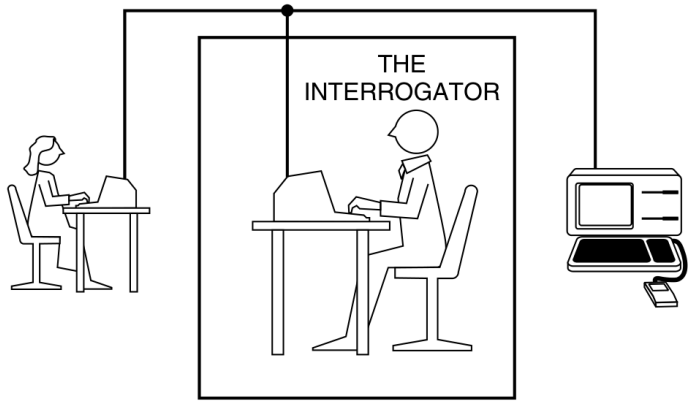


Figure 1.1 The Turing test.

Turing replaced "Can machines think?" (ambiguous) with the **imitation game**. An interrogator communicates only by text with a human and a machine. If the interrogator cannot tell which is which, the machine is assumed intelligent.

Three advantages (Luger)

1. It gives an **objective notion** of intelligence (behaviour of a known intelligent being) and avoids debates about its “true” nature.
2. It avoids unanswerable questions: does the machine use the right internal processes, is it conscious?
3. It **removes bias** toward living organisms: the interrogator sees only the content of answers.

Criticisms and replies

- Biased to **symbolic** problem solving. Does not test perception or manual dexterity.
- Forces machine intelligence into a **human mould**. A machine should exploit its own strengths (fast, reliable memory). Ford and Hayes call the full test a distraction.
- **Lady Lovelace’s objection:** computers only do what they are told. Reply: expert systems reached conclusions their designers did not anticipate.
- **Argument from informality of behaviour:** no rule set covers every situation. Reply: modern AI uses production systems, object systems, neural nets and pattern matching, which are flexible and not rigidly ordered.
- **Simon’s ant:** apparent complexity of behaviour comes from the complexity of the environment, not of the ant’s internal program. Intelligence depends on interaction with a rich environment (culture).

1.4 Agents, situated and social models of intelligence

Logic-based AI (rationalist, “GOFAI”) was challenged by Wittgenstein (meaning depends on human cultural context), Husserl, Heidegger and Merleau-Ponty (intelligence is *coping in the world*, not knowing propositions), and critics such as Dreyfus, Winograd and Flores, Searle and Brooks. Alternatives: **neural/connectionist** models, **genetic algorithms and artificial life**, and **social/agent** models (e.g. the bakeries of New York keep the city stocked with bread although no individual predicts demand).

Five themes of the agent-oriented view

Theme	Meaning
Autonomous	Each agent has its own responsibilities with little or no knowledge of what others do.
Situated	Sensitive to its own surrounding environment; knows nothing of the whole domain.
Interactional	Agents cooperate on a task like a society; knowledge and skills are distributed.
Structured	Agents coordinate, so the solution is collective and cooperative.
Emergent	The result is greater than the sum of its contributors; intelligence lives in the society.

> **Agent**
> - An element of a society that can **perceive** (often limited) aspects of its environment and **affect** that environment, directly or through cooperation with other agents.

- **Agent types:** rote agents, coordination agents, search agents, learning agents, decision agents.
- **Three requisites for building such a society:** (1) structures for representing information, (2) strategies for searching alternatives, (3) architectures that support agent interaction.

1.5 Application areas of AI

The two most fundamental concerns of AI are **knowledge representation** and **search**.

Area	Key points
Game playing	Well-defined rules make the space easy to generate; no ethical or financial cost; huge spaces need **heuristics** (useful but fallible strategies). Chess and backgammon champions; checkers fully solved in 2007.
Automated reasoning and theorem proving	Oldest branch (Logic Theorist, GPS). Logic is formal so lends itself to automation. Early provers drowned in irrelevant theorems, which led to heuristic (expert system) approaches. Modern provers act as **intelligent assistants**.
Expert systems	Power comes from **domain-specific knowledge** (theory plus heuristics) obtained from a human expert by a knowledge engineer. Examples: DENDRAL (molecular structure), MYCIN (meningitis and blood infections; uncertainty and explanation), PROSPECTOR, INTERNIST, Dipmeter Advisor, XCON (configured VAX computers).
Natural language understanding	Needs extensive background knowledge, not just parsing. Early successes in micro-worlds (SHRDLU, blocks world) did not generalise. Modern work uses stochastic models. Full understanding is still beyond the state of the art.
Modelling human performance	Programs need not copy the human mind, but explicit models help psychology (cognitive science) test theories.
Planning and robotics	Find a sequence of atomic actions to reach a goal. Hierarchical decomposition (e.g. trip planning) reduces the search. Robots must react to a changing environment. Led to agent-oriented problem solving (Brooks).
Languages and environments	Lisp, Prolog, object-oriented programming, trace packages. Many are now standard software engineering tools.
Machine learning	An expert system re-solves the same problem every time. Early programs: AM (Automated Mathematician), Winston’s arch learner, ID3, Meta-DENDRAL, Teiresias, Hacker, explanation-based learning.
Neural nets and genetic algorithms	Neurons: dendrites receive, threshold, axon fires, synapses excitatory or inhibitory. Robust to noise, naturally parallel. GAs use crossover and mutation to evolve solutions.
AI and philosophy	AI programs are experiments. Physical Symbol System Hypothesis (Newell and Simon 1976): a physical symbol system is necessary and sufficient for intelligence.

Five deficiencies of expert systems

1. Difficulty capturing “deep” knowledge (MYCIN once asked if a male patient was pregnant).
2. Lack of robustness and flexibility; cannot go back to first principles.
3. Inability to give deep explanations (only the steps taken).
4. Difficulty of verification (air traffic control, reactors, weapons).
5. Little learning from experience; handcrafted.

1.6 Summary: common features of AI (Luger’s 8 points)

1. Use of computers to do reasoning, pattern recognition, learning or other inference.
2. Focus on problems that do **not** respond to algorithmic solutions (so heuristic search).
3. Problem solving with inexact, missing or poorly defined information.
4. Reasoning about significant **qualitative** features of a situation.
5. Attempt to deal with semantic meaning as well as syntactic form.
6. Answers that are neither exact nor optimal but “sufficient”.
7. Large amounts of **domain-specific knowledge** (expert systems).
8. Use of **meta-level knowledge** for more sophisticated control of problem solving.

Exam focus

- Be able to define AI and explain why the definition is hard.
- Explain the Turing test, its three advantages and main criticisms.
- Contrast rationalist and empiricist traditions and say how each shows up in AI.
- Explain how agent-based AI differs from logic-based AI (five themes).
- List expert system deficiencies and name DENDRAL, MYCIN, XCON.

Chapter 2 — The Predicate Calculus

Predicate calculus is AI's **representation language**. It gives (1) a well-defined **syntax**, (2) a formal **semantics**, and (3) **sound and complete inference rules**. Everything later (search, expert systems, theorem proving, Prolog) builds on it.

2.1 Propositional calculus

Symbol type	Symbols
Propositional symbols	P, Q, R, S, … (statements that may be true or false)
Truth symbols	true, false (sentence symbols; *not* the same as the values T and F)
Connectives	∧ (and), ∨ (or), ¬ (not), → (implies), ≡ (equivalent)
Grouping	( ), [ ]

Sentence formation rules and terminology

Every propositional symbol and truth symbol is a sentence. The negation of a sentence is a sentence. The conjunction, disjunction, implication and equivalence of two sentences are sentences. A legal sentence is a **well-formed formula (WFF)**.

- **Conjuncts:** the parts of P ∧ Q. **Disjuncts:** the parts of P ∨ Q.
- **Premise (antecedent)** is P in P → Q; **conclusion (consequent)** is Q.
- An **interpretation** maps propositional symbols to {T, F}.

Truth tables

P	Q	¬P	P ∧ Q	P ∨ Q	P → Q	P ≡ Q	¬P ∨ Q
T	T	F	T	T	T	T	T
T	F	F	F	T	F	F	F
F	T	T	F	T	T	F	T
F	F	T	F	F	T	T	T

Columns P → Q and ¬P ∨ Q are identical, which proves material implication. Implication is false **only** when P is true and Q is false.

Logical identities (memorise)

Name	Identity
Double negation	¬¬P ≡ P
Material implication	(P → Q) ≡ (¬P ∨ Q)
Contrapositive	(P → Q) ≡ (¬Q → ¬P)
De Morgan	¬(P ∨ Q) ≡ (¬P ∧ ¬Q) ¬(P ∧ Q) ≡ (¬P ∨ ¬Q)
Commutative	(P ∧ Q) ≡ (Q ∧ P) (P ∨ Q) ≡ (Q ∨ P)
Associative	((P ∧ Q) ∧ R) ≡ (P ∧ (Q ∧ R)) ((P ∨ Q) ∨ R) ≡ (P ∨ (Q ∨ R))
Distributive	P ∨ (Q ∧ R) ≡ (P ∨ Q) ∧ (P ∨ R) P ∧ (Q ∨ R) ≡ (P ∧ Q) ∨ (P ∧ R)

Boole: AND = * (∧), OR = + (∨), NOT = ¬, with the law X*X = X (P ∧ P ≡ P); repetition does not add knowledge. Only 0 and 1 satisfy it. **Propositional calculus is decidable**: with n symbols there are 2^n interpretations to check.

2.2 Predicate calculus: syntax

Aspect	Propositional	Predicate
Atomic unit	Whole proposition P	Predicate with terms
Access to components	No	Yes
Variables / quantifiers	No	Yes (∀, ∃)
Example	P = "it rained on Tuesday"	weather(tuesday, rain)

Alphabet and symbol rules

- Allowed: upper and lower case letters, digits 0–9, underscore. Symbols must start with a letter.
- Illegal: `% @ / & * !` and spaces inside symbols. Examples of illegal symbols: `3jack, ab%cd, ***71, duck!!!, no blanks allowed`.
- Improper symbols (not objects or relations): parentheses, comma, full stop.
- Reserved: `true` and `false`.

Symbol type	Starts with	Meaning	Examples
Constant	lowercase	names a specific object	george, tree, tall, blue, fire3, tom_and_jerry
Variable	UPPERCASE	general class of objects	George, BILL, KATE, X
Function	lowercase	mapping with an arity	father, plus, price
Predicate	lowercase	relationship with an arity	likes, equals, on, near

- **Arity** = number of arguments. Predicates with the same name but different arity are **different relations**: likes(george, kate) vs likes(george, sarah, tuesday).
- **Term** = constant, variable, or function expression (function symbol of arity n followed by n terms). Examples: cat, X, times(2,3), mother(sarah).
- **Atomic sentence** = predicate of arity n followed by n terms. Example: friends(father_of(david), father_of(andrew)).
- `plus(two,three)` is a function expression, **not** a sentence. `equal(plus(two,three), five)` is an atomic sentence.
- Well-formedness and truth are independent: equal(plus(2,3), seven) is well formed but false.

The seven sentence formation rules

1. Every atomic sentence is a sentence.
2. If S is a sentence, ¬S is a sentence.
3. $S_1 \wedge S_2$ is a sentence.
4. $S_1 \vee S_2$ is a sentence.
5. $S_1 \rightarrow S_2$ is a sentence.
6. $S_1 \equiv S_2$ is a sentence.
7. If X is a variable and S a sentence, then ∀X S and ∃X S are sentences.

Quantifier	Name	Example
∀	Universal: true for all values	∀X likes(X, ice_cream): everyone likes ice cream
∃	Existential: true for at least one value	∃Y friends(Y, peter): someone is a friend of Peter

Family example

```
mother(eve, abel).   mother(eve, cain).   father(adam, abel).   father(adam, cain).
∀X ∀Y  father(X, Y) ∨ mother(X, Y)  →  parent(X, Y)
∀X ∀Y ∀Z  parent(X, Y) ∧ parent(X, Z)  →  sibling(Y, Z)
```

2.3 Semantics of predicate calculus

Interpretation over a domain D (non-empty set)

- Each **constant** is assigned an element of D. Each **variable** is assigned a non-empty subset of D. Each **function** of arity m maps $D^m \rightarrow D$. Each **predicate** of arity n maps $D^n \rightarrow \{T, F\}$.

- Value of ¬S, $S_1 \wedge S_2$, ∨, →, ≡ follow the propositional truth tables.
- ∀X S is T if S is T for **all** assignments to X. ∃X S is T if S is T for **some** assignment to X.

Term	Definition
Free variable	Not within the scope of any quantifier
Closed expression	All variables are quantified
Ground expression	Contains no variables
Dummy variable	Name can be replaced by any other without changing meaning
Scope	The instances of a variable over which a quantifier holds

Decidability

- **Propositional:** decidable (finitely many interpretations).
- **Predicate: not** decidable. Domains can be infinite, so exhaustive testing is impossible.

Quantifier equivalences

Equivalence	Reading
¬∃X p(X) ≡ ∀X ¬p(X)	“No X is p” = “all X are not p”
¬∀X p(X) ≡ ∃X ¬p(X)	“Not all X are p” = “some X is not p”
∃X p(X) ≡ ∃Y p(Y) ∀X q(X) ≡ ∀Y q(Y)	Renaming dummy variables
∀X (p(X) ∧ q(X)) ≡ ∀X p(X) ∧ ∀Y q(Y)	∀ distributes over ∧
∃X (p(X) ∨ q(X)) ≡ ∃X p(X) ∨ ∃Y q(Y)	∃ distributes over ∨

First-order predicate calculus: quantifiers range over objects only. **Higher-order**: quantifiers also range over predicates and functions, e.g. ∀(Likes) Likes(george, kate).

English to predicate calculus

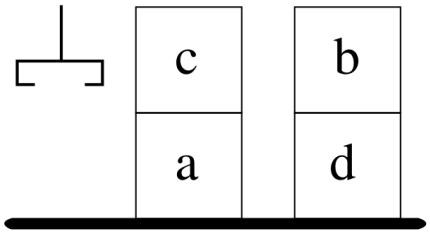
English	Predicate calculus
If it does not rain on Monday, Tom will go to the mountains.	¬weather(rain, monday) → go(tom, mountains)
Emma is a Doberman pinscher and a good dog.	gooddog(emma) ∧ isa(emma, doberman)
All basketball players are tall.	∀X (basketball_player(X) → tall(X))
Some people like anchovies.	∃X (person(X) ∧ likes(X, anchovies))
If wishes were horses, beggars would ride.	equal(wishes, horses) → ride(beggars)
Nobody likes taxes.	¬∃X likes(X, taxes)

- There is **no unique** translation; the aim is expressiveness and efficiency.
- Typical trap: “all” uses → (∀X (p(X) → q(X))) but “some” uses ∧ (∃X (p(X) ∧ q(X))).

verify_sentence

```
function verify_sentence(expression);
  case expression is an atomic sentence:           return SUCCESS;
       expression is  Q X s  (Q is ∀ or ∃):         return verify_sentence(s);
       expression is  ¬s:                            return verify_sentence(s);
       expression is  s1 op s2  (binary operator): SUCCESS only if both parts verify;
       otherwise:                                  return FAIL
```

2.4 The blocks world



on(c,a)
on(b,d)
ontable(a)
ontable(d)
clear(b)
clear(c)
hand_empty

Figure 2.3 A blocks world with its predicate calculus description.

- STATE 1: on(c,a), on(b,d), ontable(a), ontable(d), clear(b), clear(c), hand_empty.
- **Rules:** clear(X) if nothing is on X: ∀X (¬∃Y on(Y,X) → clear(X)). If Y is on the table nothing is under it. The hand is empty iff it grips nothing.
- **Declarative semantics:** no assumed order of evaluation. **Procedural semantics:** a method for evaluating expressions over time (as in Prolog). Adding it brings the **frame problem** (what does *not* change) and **non-monotonicity**.

Operator	Preconditions	Add list	Delete list
pickup(X)	gripping( ) ∧ clear(X) ∧ ontable(X)	gripping(X)	ontable(X) ∧ gripping( )
putdown(X)	gripping(X)	ontable(X) ∧ gripping( ) ∧ clear(X)	gripping(X)
stack(X,Y)	clear(Y) ∧ gripping(X)	on(X,Y) ∧ gripping( ) ∧ clear(X)	clear(Y) ∧ gripping(X)
unstack(X,Y)	clear(X) ∧ gripping( ) ∧ on(X,Y)	gripping(X) ∧ clear(Y)	gripping( ) ∧ on(X,Y)

2.5 Inference rules

Term	Definition
Satisfy	Interpretation I satisfies X if X is T under I and a particular variable assignment
Model	I is a model of X if I satisfies X for *all* variable assignments
Satisfiable	Some interpretation and assignment satisfies X
Unsatisfiable (inconsistent)	No interpretation satisfies X. Example: ∃X (p(X) ∧ ¬p(X))
Valid	X is T under *every* interpretation. Example: ∀X (p(X) ∨ ¬p(X))
Logically follows	Every interpretation satisfying S also satisfies X
Sound (rule)	Everything the rule produces logically follows from S
Complete (rule)	The rule can infer everything that logically follows from S
Proof procedure	Inference rule plus an algorithm for applying it

Rule	Form	Meaning
Modus ponens	P, P → Q ⊢ Q	From P and P implies Q, infer Q
Modus tollens	P → Q, ¬Q ⊢ ¬P	If P implies Q and not Q, infer not P
And elimination	P ∧ Q ⊢ P (and Q)	From a conjunction infer either conjunct
And introduction	P, Q ⊢ P ∧ Q	From both infer the conjunction
Universal instantiation	∀X p(X) ⊢ p(a)	Substitute a ground term for a universally quantified variable

Socrates syllogism

```
1. ∀X (man(X) → mortal(X))
2. man(socrates)
3. man(socrates) → mortal(socrates)      universal instantiation {socrates/X}
4. mortal(socrates)                        modus ponens on 2 and 3
```

Unification is what lets a program discover that {socrates/X} makes modus ponens applicable. A stronger rule, **resolution**, comes in Chapter 14.

2.6 Unification

> **Unification**
> - An algorithm that finds the **substitutions** that make two predicate calculus expressions match.

- A **constant** cannot be replaced; two different constants do not unify.
- A variable may be bound to a constant, a term, or another variable.
- **Occurs check:** a variable cannot unify with a term that contains it (X with f(X) fails).
- Substitutions must be applied **consistently** across all occurrences.
- Composition of substitutions is **associative but not commutative**.
- Return the **most general unifier (mgu)**, unique up to renaming of variables. A less general unifier (e.g. {fred/X, fred/Y}) may eliminate solutions; {Z/X, Z/Y} keeps them.
- **Skolemization** removes existential quantifiers: ∀X ∃Y mother(X,Y) becomes ∀X mother(X, f(X)) with Skolem function f.

Expression 1	Expression 2	Unifier
foo(X, a, goo(Y))	foo(fred, a, goo(Z))	{fred/X, Z/Y}
foo(X, a, goo(Y))	foo(W, a, goo(jack))	{W/X, jack/Y}
foo(X, a, goo(Y))	foo(Z, a, goo(moo(Z)))	{Z/X, moo(Z)/Y}
man(X)	man(socrates)	{socrates/X}
likes(X, Y)	likes(george, kate)	{george/X, kate/Y}
p(X)	p(Y)	{Z/X, Z/Y} (most general)
X	f(X)	FAIL (occurs check)

Composition example

```
first   {X/Y, W/Z}      second  {V/X}      third  {a/V, f(b)/W}
third ∘ second   = {a/X, a/V, f(b)/W}
result ∘ first   = {a/Y, a/X, a/V, f(b)/Z, f(b)/W}
```

The unify algorithm

```
function unify(E1, E2);
  both constants or empty list:  if E1 = E2 return {} else FAIL
  E1 is a variable:   if E1 occurs in E2 return FAIL else return {E2/E1}
  E2 is a variable:   if E2 occurs in E1 return FAIL else return {E1/E2}
  either is empty:    FAIL
  otherwise:
      SUBS1 := unify(first of E1, first of E2);   if FAIL, return FAIL
      TE1 := apply(SUBS1, rest of E1);  TE2 := apply(SUBS1, rest of E2)
      SUBS2 := unify(TE1, TE2);                   if FAIL, return FAIL
      return composition(SUBS1, SUBS2)
```

Trace: unify((parents X (father X) (mother bill)), (parents bill (father bill) Y)) gives {} for parents, {bill/X} for X, {} for (father bill), then {(mother bill)/Y}. **Answer: {bill/X, (mother bill)/Y}**.

2.7 The financial advisor (logic version)

Task: recommend savings, stocks, or a combination. Adequacy functions: **minsavings(X) = 5000 × X** and **minincome(X) = 15000 + 4000 × X** (X = number of dependents; income must also be steady).

```
1. savings_account(inadequate)                          → investment(savings)
2. savings_account(adequate) ∧ income(adequate)        → investment(stocks)
3. savings_account(adequate) ∧ income(inadequate)      → investment(combination)
4. ∀X amount_saved(X) ∧ ∃Y (dependents(Y) ∧ greater(X, minsavings(Y)))      → savings_account(adequate)
5. ∀X amount_saved(X) ∧ ∃Y (dependents(Y) ∧ ¬greater(X, minsavings(Y)))     → savings_account(inadequate)
6. ∀X earnings(X, steady) ∧ ∃Y (dependents(Y) ∧ greater(X, minincome(Y)))     → income(adequate)
7. ∀X earnings(X, steady) ∧ ∃Y (dependents(Y) ∧ ¬greater(X, minincome(Y)))    → income(inadequate)
8. ∀X earnings(X, unsteady)                                                          → income(inadequate)
```

Predicates: savings_account/1, income/1, investment/1 (stocks, savings, combination), amount_saved/1, earnings/2 (amount, steady or unsteady), dependents/1, greater/2. Rules 1–3 give the advice; rules 4–8 decide adequacy. Rule 8 is the textbook's unsteady-income rule.

Exam focus

- Write English sentences in predicate calculus (watch ∀ with → and ∃ with ∧).
- Decide whether a symbol or sentence is well formed, and why.
- Perform unification step by step; spot the occurs check and give the **most general** unifier.
- Define sound, complete, valid, satisfiable, model; apply modus ponens, modus tollens, universal instantiation.
- Why propositional calculus is decidable and predicate calculus is not.

Chapter 3 — Structures and Strategies for State Space Search

Inferences define a space that is searched for a solution. Questions search theory must answer: Is a solution guaranteed? Does it terminate? Is it optimal? What are the time and memory costs? How can complexity be reduced?

3.1 Representation and search (Part II introduction)

- AI = the study of **representation and search**. Dartmouth workshop (1956) named the field; topics included automatic computers, language, neuron nets, size of a calculation, self-improvement, abstraction, randomness and creativity.
- A representation **abstracts** the critical features of a domain. Trade-off: **expressiveness** vs **efficiency**. Examples: floating-point numbers sacrifice full expressive power to be possible at all; a pixel array suits image processing but cannot express the semantic organisation of the image.
- **State space search:** nodes are states, arcs are moves or inferences. Tic-tac-toe, car diagnosis (nodes = states of partial knowledge) both fit.
- **Exhaustive search** is impractical (chess $\approx 10^{120}$ states). **Heuristics** guide search along promising paths.

3.2 Graph theory

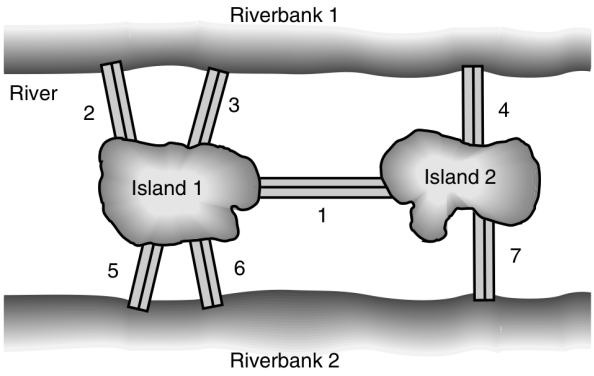


Figure 3.1 The city of Königsberg.

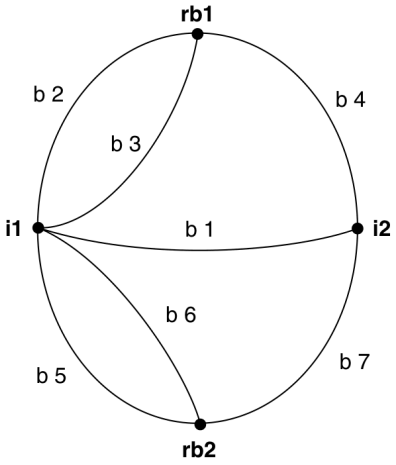
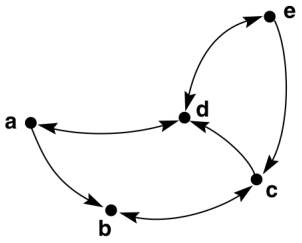


Figure 3.2 Graph of the Königsberg bridge system.

Euler solved Königsberg by reducing the map to a graph (land masses = nodes, bridges = arcs) and counting the **degree** of nodes. A walk crossing every bridge once exists only if the graph has **exactly zero or two nodes of odd degree**. Königsberg has four, so the walk is impossible (an **Euler path**). The predicate calculus version (connect(i1, i2, b1)) hides the idea of node degree; the graph makes it visible.

Term	Definition
Graph	Set of nodes N and set of arcs (ordered pairs of nodes). Nodes need not be finite.
Labelled / directed	Nodes or arcs carry descriptors / arcs have direction
Rooted graph	A unique root from which all paths originate; the root has no parent
Leaf (tip)	Node with no children
Path of length n−1	Ordered sequence [N_1, ..., N_n] with each consecutive pair an arc
Cycle (loop)	A path that contains a node more than once
Tree	Graph with a unique path between every pair of nodes (no cycles); each node has a unique parent
DAG	Directed acyclic graph (e.g. tic-tac-toe state space; moves cannot be undone)
Parent / child / sibling	Along a directed arc; siblings share a parent
Ancestor / descendant	Earlier / later node along a path
Connected	A path exists that includes both nodes



Nodes = **{a,b,c,d,e}**

Arcs = **{(a,b),(a,d),(b,c),(c,b),(c,d),(d,a),(d,e),(e,c),(e,d)}**

Figure 3.3 A labeled directed graph.

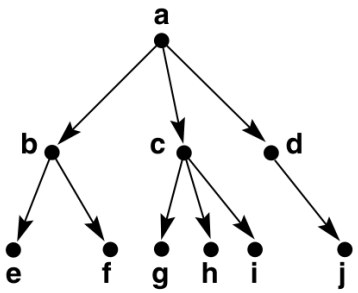


Figure 3.4 A rooted tree, exemplifying family relationships.

3.3 Finite state machines and state space definition

Finite state machine (FSM)

- An ordered triple **(S, I, F)**: S finite set of states, I finite set of inputs, F the transition function with F_i: S → S for each i ∈ I. If the machine is in S_j and i occurs, the next state is $F_i(S_j)$.

Finite state acceptor (Moore machine)

- An FSM with a start state s_0 and one or more accept states. An input stream is accepted if it ends in an accept state. Written (S, s_0, {s_n}, I, F).

Flip-flop: S = {s_0, s_1}, I = {0, 1}. Input 0 leaves the state unchanged; input 1 toggles it.

State \ input	0	1
s_0	s_0	s_1
s_1	s_1	s_0

A second machine recognises strings over {a, b, c, d} that contain the sequence “abc” (s_0 is the start state, s_3 the accepting state). Acceptors define a **formal language** and are used in natural language processing.

State space = four-tuple [N, A, S, GD]

- N = nodes (states); A = arcs (steps); S = non-empty subset of N, the start state(s); GD = non-empty subset of N, the goal state(s), described by (1) a measurable property of a **state**, or (2) a measurable property of the **path** (e.g. total cost). A **solution path** goes from a node in S to a node in GD.

- States can be reached by different paths, so general graph search must **detect loops**. Tree search can skip that test.
- **Tic-tac-toe:** a DAG with 9! = 362,880 paths. Chess has 10^{120}, checkers 10^{40} paths.

- **8-puzzle:** move the *blank* (up, right, down, left). It is a graph with cycles. The full space of the 8/15-puzzle is two disconnected, equal-size subgraphs (half of all states are unreachable).
- **Travelling salesperson:** goal is a property of the whole path (type 2). Exhaustive search is (N−1)!. **Branch and bound** prunes paths costing more than the best circuit so far (still exponential, about 1.26^n). **Nearest neighbour** (greedy, one path) is fast but fallible.

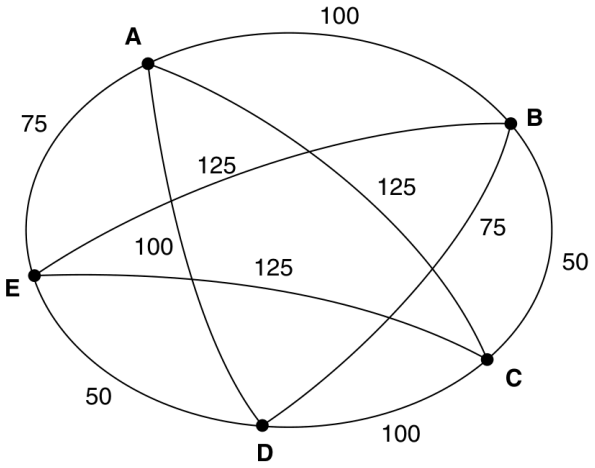


Figure 3.9 An instance of the traveling salesperson problem.

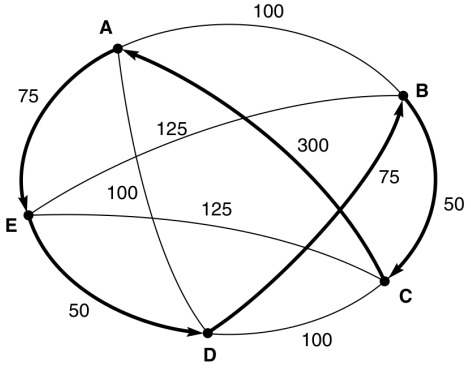


Figure 3.11 An instance of the traveling salesperson problem with the nearest neighbor path in bold. Note that this path (A, E, D, B, C, A), at a cost of 550, is not the shortest path. The comparatively high cost of arc (C, A) defeated the heuristic.

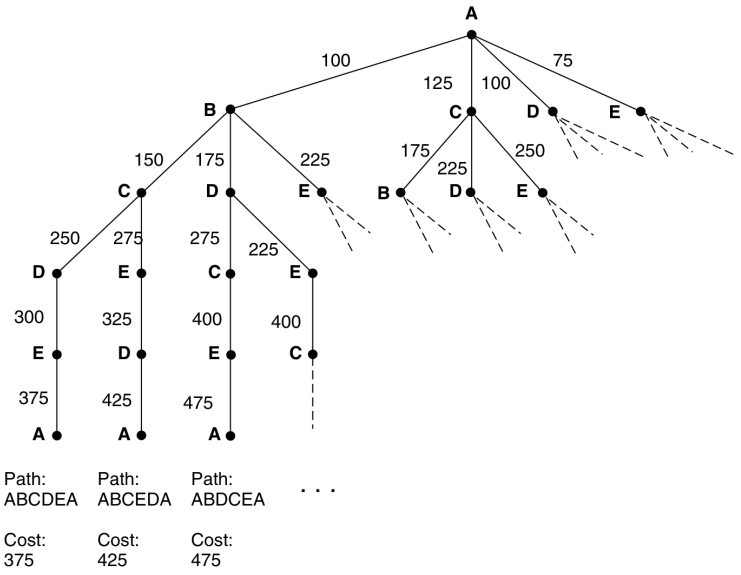


Figure 3.10 Search of the traveling salesperson problem. Each arc is marked with the total weight of all paths from the start node (A) to its endpoint.

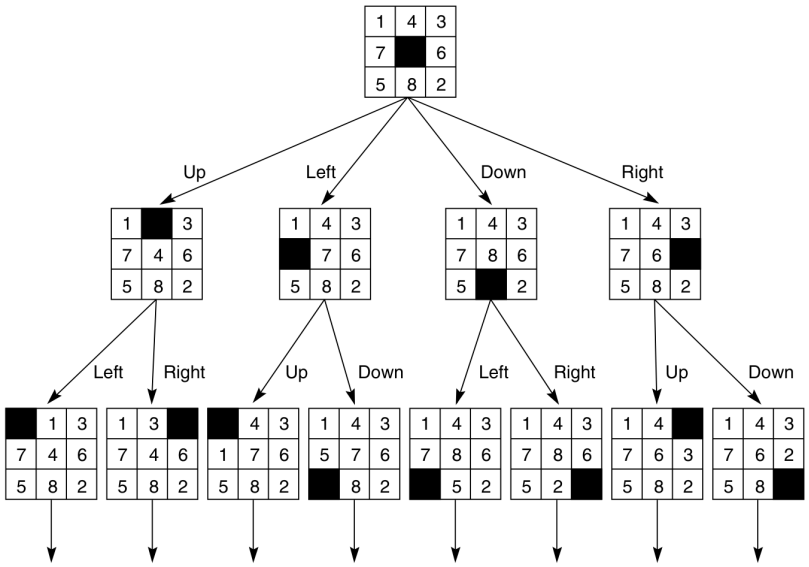


Figure 3.8 State space of the 8-puzzle generated by move blank operations.

3.4 Data-driven versus goal-driven search

	Data-driven (forward chaining)	Goal-driven (backward chaining)
Start	Given facts	Goal or hypothesis
Method	Apply rules to facts to produce new facts until the goal is reached	Find rules whose conclusion matches the goal; their premises become subgoals; work back to the facts
Suited when	**All or most data are given** (interpretation); many possible goals but few ways to use the data; hard to form a goal (DENDRAL, PROSPECTOR)	**A goal is given** or easy to formulate (theorem proving, diagnosis); many rules match the facts and fan out; data must be acquired (doctor orders only needed tests)

Both search the same graph but visit different states. Example: “I am a descendant of Thomas Jefferson”: searching back from “I” examines about 2^{10} ancestors; searching forward from Jefferson about 3^{10} descendants (both exponential, backward is smaller). The best choice depends on rule complexity, shape of the space and availability of data.

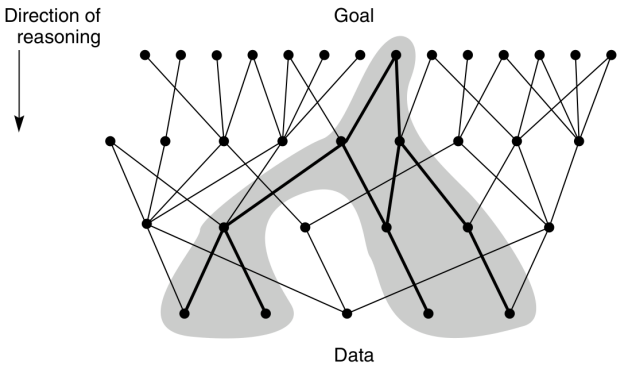


Figure 3.12 State space in which goal-directed search effectively prunes extraneous search paths.

3.5 Implementing graph search

Backtrack

Pursue a path until a goal or a dead end. At a dead end, return to the most recent node with unexamined siblings. Three lists:

List	Meaning
SL (state list)	States on the current path; returned as the solution if a goal is found
NSL (new state list)	Nodes awaiting evaluation (children not yet generated)
DE (dead ends)	States whose descendants contain no goal; never revisited

Each new state is tested for membership in SL, NSL and DE to prevent loops.

```
function backtrack;
  SL := [Start]; NSL := [Start]; DE := []; CS := Start;
  while NSL ≠ [] do
    if CS = goal then return SL;
    if CS has no children (excluding nodes on DE, SL, NSL) then
        while SL not empty and CS = first(SL) do
            add CS to DE; remove first(SL); remove first(NSL); CS := first(NSL);
        add CS to SL
    else  place children of CS (not on DE, SL, NSL) on NSL;
          CS := first(NSL); add CS to SL
  return FAIL
```

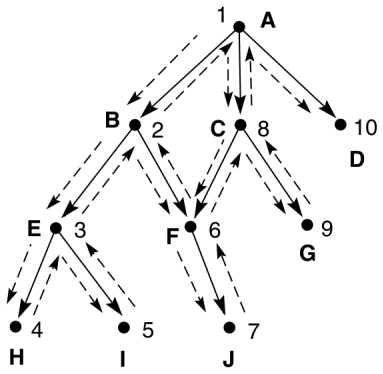


Figure 3.14 Backtracking search of a hypothetical state space.

Iter	CS	SL	NSL	DE
0	A	[A]	[A]	[ ]
1	B	[B A]	[B C D A]	[ ]
2	E	[E B A]	[E F B C D A]	[ ]
3	H	[H E B A]	[H I E F B C D A]	[ ]
4	I	[I E B A]	[I E F B C D A]	[H]
5	F	[F B A]	[F B C D A]	[E I H]
6	J	[J F B A]	[J F B C D A]	[E I H]
7	C	[C A]	[C D A]	[B F J E I H]
8	G	[G C A]	[G C D A]	[B F J E I H]

Breadth-first and depth-first search

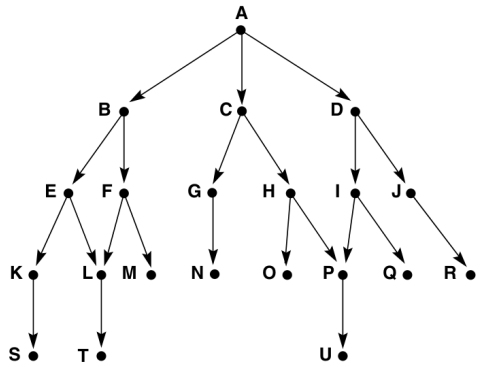


Figure 3.15 Graph for breadth- and depth-first search examples.

Depth-first visit order on this graph: A B E K S L T F M C G N H O P U D I Q J R. **Breadth-first**: A B C D E F G H I J K L M N O P Q R S T U.

	Breadth-first	**Depth-first**
open is a	**queue** (FIFO): add children on the right	**stack** (LIFO): add children on the left
Order	Level by level	Deepest first
Shortest path?	**Yes** (every state first reached along its shortest path)	**No**; may find a longer path first
Space	Exponential: B^n states on level n	Linear in path length
Risk	Combinatorial explosion in memory	Can get lost deep, or follow an infinite path
Good when	Shortest path matters; solution shallow	Solution path is long; many branches; memory limited

```
function breadth_first_search;              (depth_first_search is identical except that
  open := [Start]; closed := []             children go on the LEFT end of open - a stack)
  while open ≠ [] do
      remove leftmost state from open, call it X;
      if X is a goal then return SUCCESS
      else generate children of X; put X on closed;
           discard children already on open or closed;      % loop check
           put remaining children on RIGHT end of open      % queue
  return FAIL
```

- **open** = frontier (generated, not yet examined); **closed** = states already examined (the union of DE and SL).
- To return the path, store (state, parent) pairs, e.g. open = [(D,A), (E,B), ...]; trace back from the goal.
- In depth-first search, to keep the shortest version of a state store (state, parent, path length).

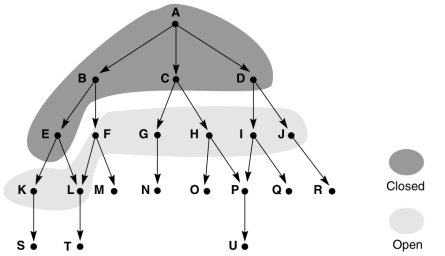


Figure 3.16 Graph of Figure 3.15 at iteration 6 of breadth-first search. States on open and closed are highlighted.

Depth-first iterative deepening (Korf 1987)

- Run depth-first with depth bound 1, then 2, 3, … No information kept between iterations.
- Finds the **shortest path** like breadth-first, with **linear space** (B × n) like depth-first. Time is still $O(B^n)$ because almost all work is at the deepest level.
- All uninformed searches have worst-case exponential time. Only **heuristics** (Chapter 4) reduce it.

3.6 State space for logic: and/or graphs

Inference rules create the arcs between states. Modus ponens search is data-driven when it proceeds from known facts and goal-driven when it works back from the proposition to be proved. The soundness and completeness of the rules guarantee correct conclusions.

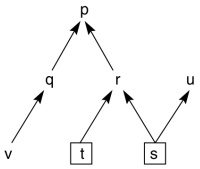


Figure 3.20 State space graph of a set of implications in the propositional calculus.

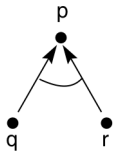


Figure 3.21 And/or graph of the expression q ∧ r ⟶ p.

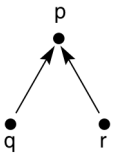


Figure 3.22 And/or graph of the expression q ∨ r ⟶ p.

- **And node** (q ∧ r → p): *all* children must be solved; arcs joined by a curved link. Means **problem decomposition**.
- **Or node** (q ∨ r → p): *any* child suffices. Means a **selection** of strategy.
- **Hypergraph:** (N, H) where each hyperarc is (a node, a subset of N). A **k-connector** has k descendants; k = 1 is an or arc, k > 1 an and arc. An ordinary graph is a hypergraph with all |descendants| = 1.

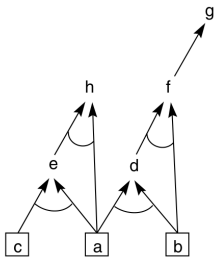


Figure 3.23 And/or graph of a set of propositional calculus expressions.

Facts a, b, c; rules a∧b→d, a∧c→e, b∧d→f, f→g, a∧e→h. **Goal-driven proof of h:** prove a and e; e needs c and a, both given. **Data-driven:** from a, b, c derive d and e, then f and h.

Example: goal-driven search with unification (fred and sam)

```
1 collie(fred).   2 master(fred, sam).   3 day(saturday).   4 ¬warm(saturday).   5 trained(fred).
6 ∀X [spaniel(X) ∨ (collie(X) ∧ trained(X)) → gooddog(X)]
7 ∀X,Y,Z [gooddog(X) ∧ master(X,Y) ∧ location(Y,Z) → location(X,Z)]
8 (day(saturday) ∧ warm(saturday))  → location(sam, park)
9 (day(saturday) ∧ ¬warm(saturday)) → location(sam, museum)
Goal: ∃X location(fred, X)
```

Rule 7 unifies with {fred/X, X/Z}, giving subgoals gooddog(fred) ∧ master(fred,Y) ∧ location(Y,X). gooddog(fred) follows from clauses 1, 5 and 6 (spaniel(fred) is assumed false). master(fred,Y) gives {sam/Y}. location(sam,X) fails with rules 7 and 8 and succeeds with rule 9. **Answer: location(fred, museum).** The order of clauses affects search order, and variables have no global scope.

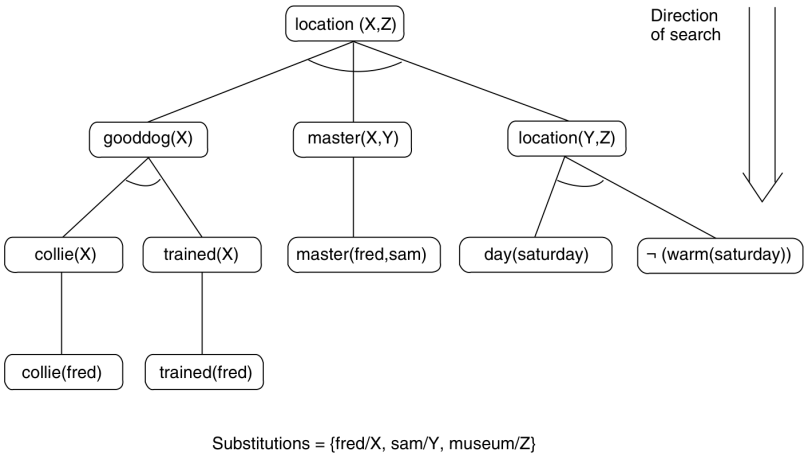


Figure 3.25 The solution subgraph showing that fred is at the museum.

Example: financial advisor revisited

Goal ∃X investment(X), depth-first with backtracking. With 2 dependents, $20,000 saved and $30,000 steady income: rule 1 needs savings_account(inadequate) (rule 5), but greater(20000, minsavings(2)) = greater(20000, 10000) is true, so the *inadequate* branch fails and search backtracks. Rule 2 then succeeds: savings_account(adequate) (rule 4) and income(adequate) (rule 6, minincome(2) = 23000 < 30000), giving **investment(stocks)**. The program asks the user for data only when no rule concludes a subgoal.

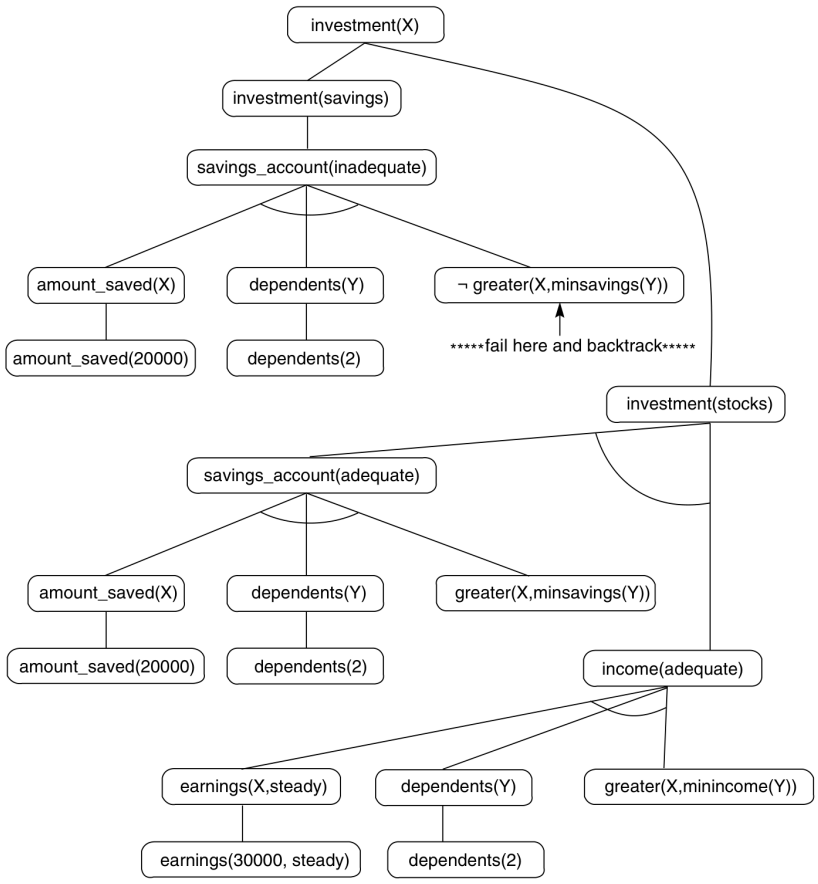


Figure 3.26 And/or graph searched by the financial advisor.

Example: English grammar as an and/or graph

```
1 sentence → np vp          2 np → n            3 np → art n       4 vp → v        5 vp → v np
6 art → a   7 art → the   8 n → man   9 n → dog   10 v → likes   11 v → bites
```

- Left-hand sides are and nodes; several rules with the same left side form or nodes; the words are the **terminals**.
- **Parsing** (data-driven): rewrite words up to *sentence*. “the dog bites the man” reduces via rules 7, 7, 8, 3, 9, 3, 11, 5, 1.
- **Generation** (goal-driven): expand *sentence* down to terminals (about 80 legal sentences; syntax only, so “the man bites the dog” is accepted).
- This keep-and-match rewriting is a **production system** (Chapter 6).

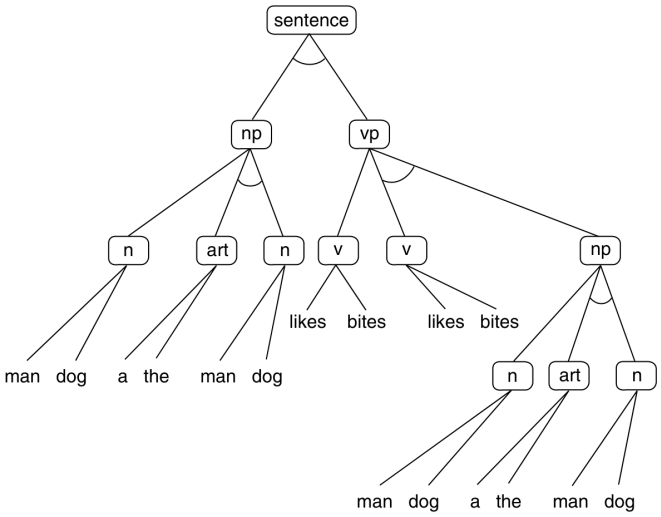


Figure 3.27 And/or graph for the grammar of Example 3.3.6. Some of the nodes (np, art, etc.) have been written more than once to simplify drawing the graph.

Exam focus

- Define graph, tree, DAG, rooted graph, path, cycle; Euler’s odd-degree rule.
- State the state space four-tuple and goal types 1 and 2.
- Trace backtrack (SL, NSL, DE, CS) and trace BFS/DFS with open and closed.
- Compare BFS, DFS and iterative deepening on shortest path, space and time.
- Choose data-driven or goal-driven for a scenario and justify it.
- Draw and/or graphs; solve a goal with unification (fred, financial advisor).

Chapter 4 — Heuristic Search

"Intelligence for a system with limited processing resources consists in making wise choices of what to do next." (Newell and Simon, 1976). Chapter 3 showed exhaustive search is impossible; heuristics make search tractable.

4.1 Heuristics

Heuristic

- From the Greek *eurisco*, "I discover". A **rule for choosing the branches** in a state space most likely to lead to an acceptable solution. It is an informed guess, **fallible**: it can give a suboptimal solution or no solution. This is inherent and cannot be removed by "better" heuristics.

Two situations need heuristics: (1) the problem has no exact solution because of ambiguity in the statement or data (medical diagnosis); (2) an exact solution exists but is too costly to compute (chess).

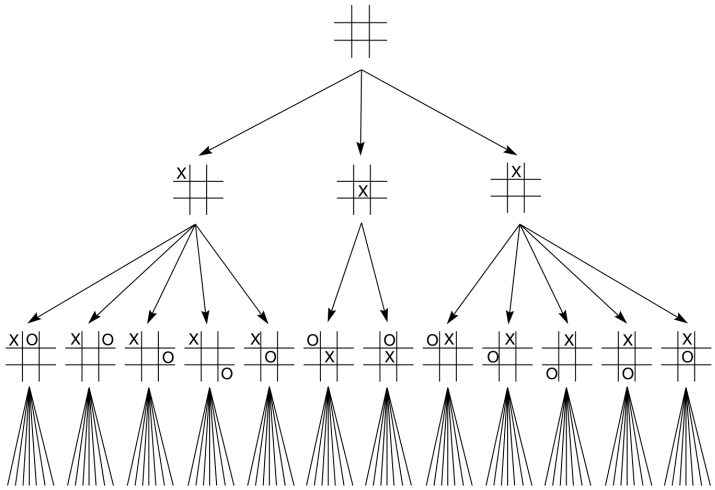


Figure 4.1 First three levels of the tic-tac-toe state space reduced by symmetry.

- **Symmetry reduction:** tic-tac-toe has 3 distinct first moves (corner, side, centre), cutting the space to 12 × 7!.
- **"Most wins" heuristic:** move to the state in which X has most winning lines. Taking the centre prunes two-thirds of the space; the crude bound is 25 states, four orders of magnitude below 9!.

4.2 Hill climbing and dynamic programming

Hill climbing

- Expand the current state, evaluate its children, pick the **best**; neither siblings nor parent are kept. No history, so it cannot recover from failure.
- **Local maximum problem:** a state better than all its children that is not the goal. Example: in the 8-puzzle, moving a tile into place may require undoing others. Look-ahead does not guarantee success (Fig 4.4).
- **Samuel's checkers (1959):** evaluation = $\Sigma\, a_i x_i$ (piece advantage, centre control, etc.), weights a_i adjusted after losses and wins (hill climbing applied to *learning*); used minimax look-ahead. Weaknesses: limited global strategy and sensitivity to erratic opponents.

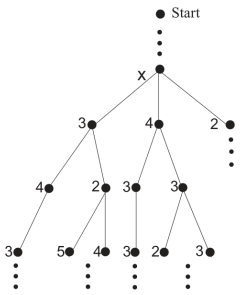


Figure 4.4 The local maximum problem for hill-climbing with 3-level look ahead.

Dynamic programming (Bellman 1956)

Also called forward–backward or (with probabilities) the **Viterbi** algorithm. It saves and **reuses solutions to subproblems** (memoisation) in an array. Cost about **n^2** (worst case n^3) versus exponential (2^n to 3^n) for exhaustive comparison. Used for spelling correction, string alignment and speech.

- **Forward stage:** fill the array from the top-left. Cell (x, y) = minimum of: cell above + 1 (insert), cell left + 1 (shift or delete), diagonal + 0 if the characters match, otherwise + 2.
- **Backward stage:** from the final cell trace back through the predecessors that produced each value to read off one optimal alignment (several may exist).
- **Minimum edit distance (Levenshtein):** insertion = 1, deletion = 1, replacement = 2 (a deletion plus an insertion). For spell checking only the forward stage is needed.

	—	B	A	A	D	D	C	A	B	D	D	A
—	0	1	2	3	4	5	6	7	8	9	10	11
B	1	0	1	2	3	4	5	6	7	8	9	10
B	2	1	2	3	4	5	6	7	6	7	8	9
A	3	2	1	2	3	4	5	6	7	8	9	8
D	4	3	2	3	2	3	4	5	6	7	8	9
C	5	4	3	4	3	4	3	4	5	6	7	8
B	6	5	6	5	4	5	4	5	4	5	6	7
A	7	6	5	4	5	6	5	4	5	6	7	6

Figure 4.6 The completed array reflecting the maximum alignment information for the strings.

	—	e	x	e	c	u	t	i	o	n
—	0	1	2	3	4	5	6	7	8	9
i	1	**2**	3	4	5	6	7	8	9	10
n	2	3	**4**	5	6	7	8	9	10	11
t	3	4	**5**	6	7	8	9	10	11	12
e	4	5	6	**5**	**6**	7	8	9	10	11
n	5	6	7	6	7	**8**	9	10	11	12
t	6	7	8	7	8	9	**8**	9	10	11
i	7	8	9	8	9	10	9	**8**	9	10
o	8	9	10	9	10	11	10	9	**8**	9
n	9	10	11	10	11	12	11	10	9	**8**

Figure 4.9 Complete array of minimum edit difference between intention and execution (adapted from Jurafsky and Martin 2000).

intention → execution costs 8: delete i (1), n→e (2), t→x (2), insert u (1), n→c (2).

```
function dynamic(source, sl, target, tl);          % minimum edit distance
  cost(0,0) := 0
  for i := 1 to sl+1: for j := 1 to tl+1:
      cost(i,j) := min[ cost(i-1, j)   + insertion cost,       % add 1
                        cost(i-1, j-1) + replace cost,         % add 2 (0 if equal)
                        cost(i, j-1)   + delete cost ]         % add 1
```

- Prune: good solutions lie near the diagonal; abandon paths whose cost passes a threshold.

4.3 Best-first search

Like depth-first and breadth-first search but **open is a priority queue ordered by heuristic merit** (best leftmost). It can recover from bad heuristic choices by returning to the next-best state on open.

```
function best_first_search;
  open := [Start]; closed := [];
  while open ≠ [] do
      remove leftmost state from open, call it X;
      if X = goal then return the path from Start to X
      else generate children of X;
           for each child:
               not on open or closed:    assign heuristic value; add to open
               already on open:          if reached by a shorter path, give it the shorter path
               already on closed:        if reached by a shorter path, remove from closed; add to open
           put X on closed;
           re-order open by heuristic merit (best leftmost)
  return FAIL
```

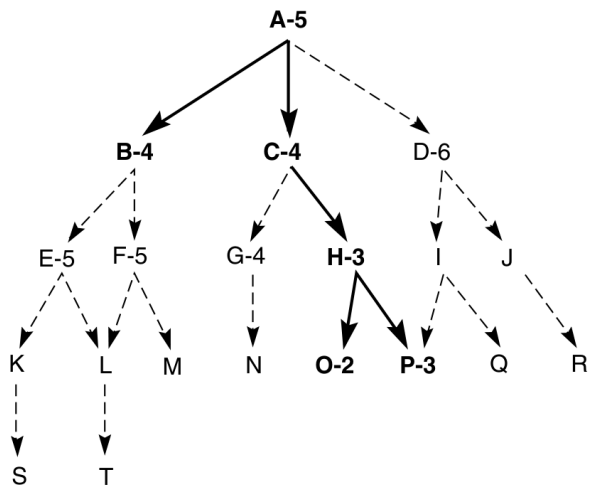


Figure 4.10 Heuristic search of a hypothetical state space.

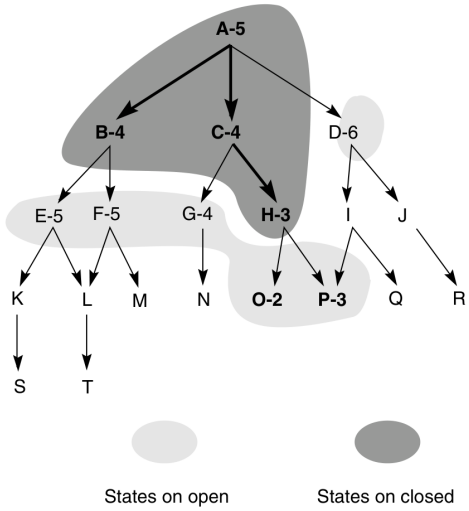


Figure 4.11 Heuristic search of a hypothetical state space with open and closed states highlighted.

- Trace: open = [A5] → [B4,C4,D6] → [C4,E5,F5,D6] → [H3,G4,E5,F5,D6] → [O2,P3,…] → [P3,…] → P is the goal. The frontier is highly uneven: best-first is opportunistic.

Evaluation functions for the 8-puzzle

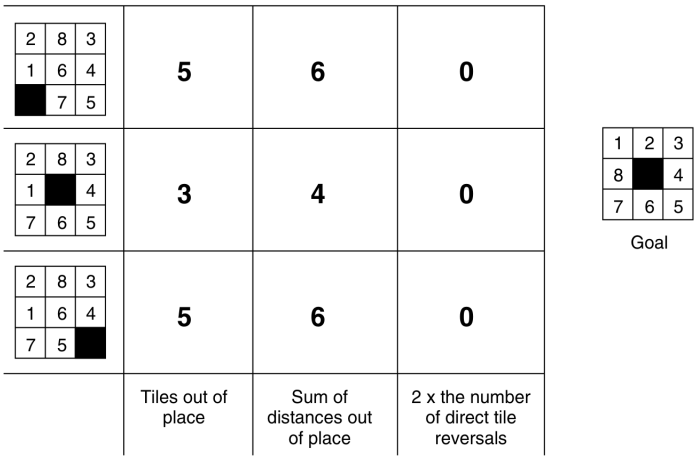


Figure 4.14 Three heuristics applied to states in the 8-puzzle.

Heuristic	Idea	Admissible?
Tiles out of place	Count of tiles not in goal position	Yes (each misplaced tile needs at least 1 move)
Sum of distances	Sum over tiles of squares to move	Yes (more informed)
2 × direct reversals	Penalise adjacent tiles needing exchange	Poor alone: gives 0 on all three states
Sum of distances + 2 × reversals	Combines the two above	Fixes the weakness of the reversal heuristic

f(n) = g(n) + h(n)

- **g(n)** = actual path length from the start to n (depth). **h(n)** = heuristic estimate of distance from n to a goal. f estimates the total cost of a path through n. g gives search a **breadth-first flavour** and stops it persisting on an infinite fruitless branch. With ties in h, prefer the shallower state.

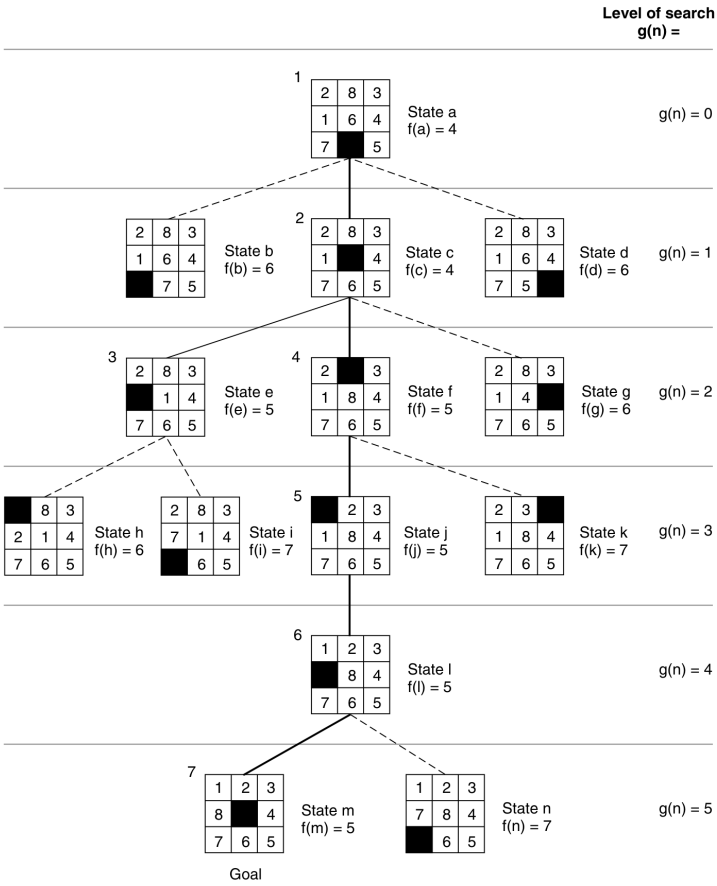


Figure 4.16 State space generated in heuristic search of the 8-puzzle graph.

Heuristics are designed **empirically**: the final measure is performance on real problem instances. In expert systems the rules themselves are heuristic; **certainty factors** (e.g. 0.8 for stocks) can be attached to conclusions and used to prune (e.g. ignore conclusions below 0.2).

4.4 Admissibility, monotonicity, informedness

Admissible

- A search algorithm is admissible if, for any graph, it always terminates with the **optimal (shortest) path** whenever a path to a goal exists. Breadth-first search is admissible.

Algorithm A and A*

- Algorithm A: best-first search with f(n) = g(n) + h(n). If **h(n) ≤ h*(n)** for all n (h never overestimates the true cost h*), it is **algorithm A***. **All A* algorithms are admissible.** Breadth-first search is A* with h = 0.

- f*(n) = g*(n) + h*(n): the actual cost of the optimal path through n. Oracles for f* do not exist, so we want f close to f*. Note g(n) ≥ g*(n).

Monotonicity (consistency)

- h is monotone if (1) for every state n_i and descendant n_j: **$h(n_i) - h(n_j) \le cost(n_i, n_j)$**, and (2) h(Goal) = 0. The heuristic is locally consistent: each state is first reached along the shortest path, so a rediscovered state can be dropped without checking path length. f never decreases.

Proof that monotone ⇒ admissible: along a path $s_1 \dots s_g$ add the inequalities $h(s_i) - h(s_{i+1}) \le cost(s_i, s_{i+1})$. The left sides telescope to $h(s_1) - h(s_g) = h(s_1)$, giving $h(s_1) \le cost(s_1, s_g)$. (Whether admissible implies monotone is left as an exercise; in general it does not.)

Informedness

- For two A* heuristics, if $\mathbf{h_1(n) \le h_2(n)}$ for all n, then h_2 is **more informed**. Chain for the 8-puzzle: 0 (breadth-first) ≤ tiles out of place ≤ sum of distances ≤ h*. A more informed A* expands a **subset** of the states of a less informed one. But the extra cost of computing the heuristic must not outweigh the saving (computer chess: simple heuristics + deep search vs sophisticated heuristics + shallow search).

4.5 Heuristics in two-person games: minimax

The opponent is hostile and unpredictable. **MAX** tries to maximise his advantage; **MIN** tries to minimise MAX's score and is assumed to have the same knowledge. Leaf values (1 = MAX wins, 0 = MIN wins) are propagated up: **MAX node takes the maximum of its children; MIN node takes the minimum.**

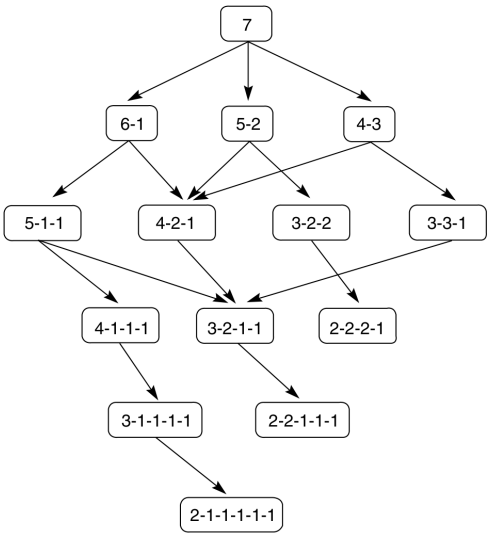


Figure 4.19 State space for a variant of nim. Each state partitions the seven matches into one or more piles.

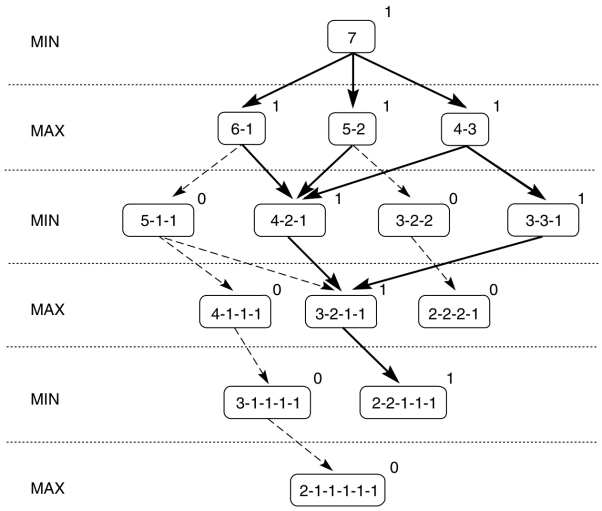


Figure 4.20 Exhaustive minimax for the game of nim. Bold lines indicate forced win for MAX. Each node is marked with its derived value (0 or 1) under minimax.

Nim (7 tokens): divide a pile into two *unequal* non-empty piles; the player who cannot move loses. Exhaustive minimax shows the second player (MAX) can always force a win.

Minimax to a fixed ply depth

- Search n plies (limited by time and memory); evaluate the leaves with a **heuristic**; back values up with minimax. The value is the heuristic value of the best state reachable in n moves, not a guaranteed win.
- Typical evaluation: (number of lines open to MAX) − (lines open to MIN) in tic-tac-toe; +∞ for a forced win, −∞ for a forced loss. Chess: piece values and position.
- **Horizon effect:** a fixed look-ahead may miss that an attractive path leads to disaster just beyond the horizon (a rook offered as bait for a queen). Mitigation: search deeper from exceptionally promising states; it never disappears.
- Deep evaluations can be biased: the minimax of estimates differs from the estimate of minimax (Pearl).

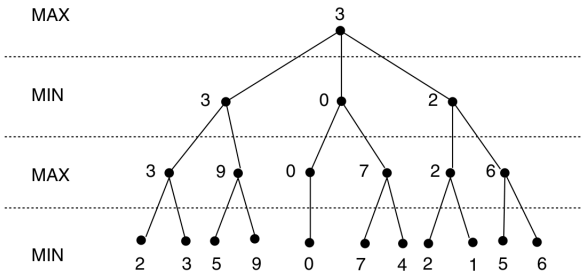


Figure 4.21 Minimax to a hypothetical state space. Leaf states show heuristic values; internal states show backed-up values.

Alpha-beta pruning

- Depth-first minimax with two values: **α** (at MAX nodes) can never decrease; **β** (at MIN nodes) can never increase. Same backed-up value as minimax with fewer nodes, in **one pass**.
- **Rule 1:** stop below a MIN node whose β ≤ the α of any MAX ancestor.
- **Rule 2:** stop below a MAX node whose α ≥ the β of any MIN ancestor.
- It relates nodes at ply n and ply n + 2, eliminating whole subtrees at ply n + 1. With fortunate ordering it can **double the searchable depth**; worst case it examines as much as plain minimax.

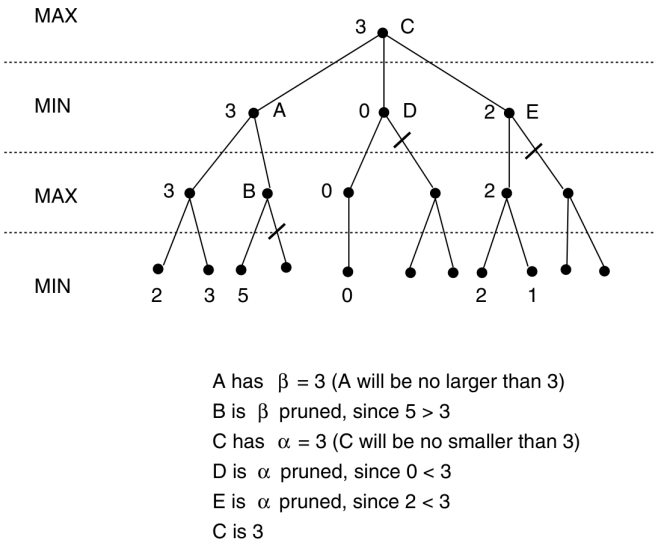


Figure 4.26 Alpha-beta pruning applied to state space of Figure 4.21. States without numbers are not evaluated.

4.6 Complexity

Branching factor and total states

- B = average number of children per state. Number of states at depth L is B^L. Total generated: $\mathbf{T = B + B^2 + \ldots + B^L = B(B^L - 1)/(B - 1)}$.

- Example: B = 2 needs about 100 states for L = 6 and about 10,000 for L = 12. Cutting B to 1.5 doubles the reachable depth for the same effort.
- **8-puzzle branching factor:** corners 2 moves (8 total), side centres 3 (12), centre 4 = 24 moves over 9 blank positions = **2.67**. Removing the move back to the parent gives **1.67**.
- **Beam search:** bound the size of open, keeping only the few best states (risk of discarding the only solution).
- The combinatorial explosion often happens unnoticed; “more powerful computers” is usually the wrong answer.
- **NP-hard** problems cannot be solved in less than exponential time without heuristics; almost all interesting search problems are in this class.
- **And/or graphs:** the estimate for an and node is at least the sum of the estimates of its children; order of and children matters (most costly, most likely to fail, expert’s choice).
- **IDA*** (Korf) combines iterative deepening with A* for linear space.

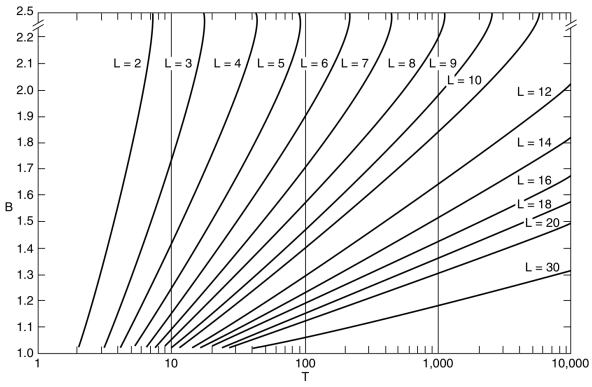


Figure 4.27 Number of nodes generated as a function of branching factor, B, for various lengths, L, of solution paths. The relating equation is: $T = B(B^L - 1)/(B - 1)$, adapted from Nilsson (1980).

Exam focus

- Define heuristic; give examples of heuristics and why they can fail.
- Trace best-first search (open/closed) and compute f = g + h for the 8-puzzle.
- Define admissible, A*, monotone, more informed; prove monotone ⇒ admissible.
- Perform minimax and alpha-beta on a tree (left-to-right and right-to-left give different prunings).
- Explain hill climbing local maxima, the horizon effect, beam search.
- Fill a dynamic programming edit-distance table and read off the edits.
- Use $T = B(B^L - 1)/(B - 1)$.

Chapter 5 — Stochastic Methods

Probabilistic reasoning suits problems with no exact or fully deterministic cause–effect relationships: **diagnosis** (the same symptoms have several causes), **natural language** (learned from a corpus), gambling and **learning** from data.

5.1 Counting

- |A| = cardinality (number of elements). Complement $\bar{A}$ = elements of the universe not in A. A ⊆ B: every element of A is in B. Empty set {} ⊆ every set.
- **Addition rule:** |A ∪ C| = |A| + |C| − |A ∩ C|. For three sets: |A ∪ B ∪ C| = |A| + |B| + |C| − |A∩B| − |A∩C| − |B∩C| + |A∩B∩C| (inclusion–exclusion).
- **Multiplication principle:** sets of size a and b give a × b pairings. Cartesian product: A × B = {(a, b) | a ∈ A ∧ b ∈ B}, |A × B| = |A| × |B|.

Quantity	Formula	Meaning
Permutations of n	n!	Ordered arrangements of all n elements
Permutations, r at a time	$_nP_r$ = n! / (n − r)!	Order matters (books on a shelf)
Combinations, r at a time	$_nC_r$ = n! / [(n − r)! r!]	Order does not matter (committee, poker hand)

Combinations = permutations divided by r! (each subset of r elements has r! orderings).

5.2 Elements of probability

Term	Definition
Elementary (atomic) event	A happening that cannot be made up of other events
Event E	A set of elementary events
Sample space S	Set of all possible outcomes (the universe)
Probability	**p(E) = |E| / |S|**

- **Results:** 0 ≤ p(E) ≤ 1; p(S) = 1; complement p($\bar{E}$) = 1 − p(E); p({}) = 0; **p(A ∪ B) = p(A) + p(B) − p(A ∩ B)**.
- **Kolmogorov's three axioms:** (1) 0 ≤ p(E) ≤ 1; (2) p(S) = 1; (3) p(A ∪ B) = p(A) + p(B) − p(A ∩ B).
- **Dice:** two fair dice give 36 outcomes. p(7) = 6/36 = 1/6; p(11) = 2/36 = 1/18; p(7 or 11) = 2/9.
- **Four of a kind:** $_{13}C_1 \times {}_4C_4 \times {}_{48}C_1 / {}_{52}C_5 = 13 \times 48 / 2{,}598{,}960 = 0.00024$.
- Complement trick: p(at least one 1 in an n-bit string) = $1 - 2^{-n}$.

Independent events

- A and B are independent iff **p(A ∩ B) = p(A) × p(B)**. When p(B) ≠ 0 this means p(A) = p(A | B): knowing B does not change the probability of A.

Example: 4-bit strings (16 total). "Even number of 1s" (8) and "ends with 0" (8) overlap in 4: 4/16 = 8/16 × 8/16, so they are **independent**. "Even number of 1s" and "ends with 1" overlap in 4 as well: p = 4/16 = 1/4 = 1/2 × 1/2, so these are also independent (exercise 12).

Probabilistic inference: the traffic example

Boolean variables: S = traffic slowdown, C = construction, A = accident. Joint distribution (consistent with the Venn diagram, Fig 5.1):

S	C	A	p
t	t	t	.01
t	t	f	.03
t	f	t	.16
t	f	f	.12
f	t	t	.01
f	t	f	.05
f	f	t	.01
f	f	f	.61

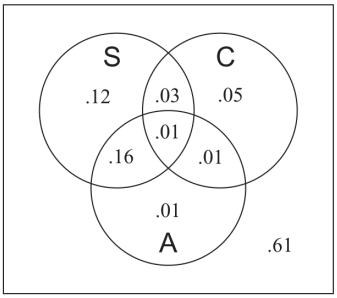


Figure 5.1 A Venn diagram representation of the probability distributions of Table 5.1; S is traffic slowdown, A is accident, C is construction.

- The probabilities sum to 1.0. **Marginalisation:** sum out the other variables. p(S = t) = .01 + .03 + .16 + .12 = **0.32**. p(C ∧ ¬S) = .01 + .05 = 0.06.

Random variables

Random variable

- A function whose domain is a sample space and whose range is a set of outcomes, usually real numbers. **Boolean** (Bernoulli trial): to {true, false} or {0, 1}. **Discrete**: countable set of values (e.g. Season = spring with p = .75). **Continuous**: range is the real numbers.

- **Binomial distribution** (r successes in n trials, success probability p): $nCr \times p^r \times (1-p)^{n-r}$. Example: 5 heads in 7 fair flips = ${}_7C_5\ (1/2)^5\ (1/2)^2$.
- **Expectation:** ex(E) = r × p(E) + c × (1 − p(E)). Roulette (37 slots, bet \$1 to win \$35): 35(1/37) + (−1)(36/37) ≈ −0.027 per play.
- **Parametric** approach: the shape of the distribution is known (fair coin, binomial); fewer data needed. **Non-parametric**: probabilities estimated by sampling (language corpus, diagnosis); needs a lot of data.

Conditional probability

- **Prior** p(event): probability before new evidence. **Posterior** p(event | evidence): probability after evidence.
- **Conditional probability:** p(d | s) = p(d ∩ s) / p(s).

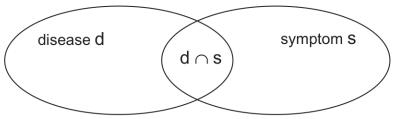


Figure 5.2 A Venn diagram illustrating the calculations of p(d|s) as a function of p(s|d).

Bayes' rule (one disease, one symptom)

- From p(s ∩ d) = p(s | d) p(d) and p(d | s) = p(d ∩ s)/p(s): **p(d | s) = p(s | d) p(d) / p(s)**. It turns a hard-to-measure posterior into easier likelihoods (e.g. count meningitis patients who have headaches, not headache sufferers who have meningitis).

Chain rule

- $p(A_1 \cap A_2 \cap \ldots \cap A_n) = p(A_1)\, p(A_2 \mid A_1)\, p(A_3 \mid A_1 \cap A_2) \ldots p(A_n \mid A_1 \cap \ldots \cap A_{n-1})$. Proved by induction, repeatedly using $p(X \cap Y) = p(X)\, p(Y \mid X)$.

Conditional independence

- A and B are conditionally independent given C iff $\mathbf{p((A \cap B) \mid C) = p(A \mid C)\, p(B \mid C)}$. Example: slowdown and orange barrels are both caused by construction, so construction *separates* them. Conditional independence shrinks joint distributions (more in d-separation, Ch. 9).

5.3 Bayes' theorem

Bayes (published 1763, after his death) relates cause and effect: from the effect we learn the probability of its causes.

$$\mathbf{p(h_i \mid E) = p(E \mid h_i)\, p(h_i) \,/\, \Sigma_{k=1..n}\, p(E \mid h_k)\, p(h_k)}$$

- $p(h_i \mid E)$ = probability hypothesis h_i is true given evidence E. $p(h_i)$ = prior. $p(E \mid h_i)$ = likelihood of the evidence under h_i.
- The denominator p(E) is a **normalising factor**, valid when the hypotheses h_i **partition** the sample space (disjoint, union = everything).
- **Maximum a posteriori** hypothesis: $\arg\max_{h_i} p(E \mid h_i)\, p(h_i)$. The denominator can be dropped when only comparing hypotheses.
- **Naive Bayes (Bayes classifier):** assume the evidence pieces are independent given the hypothesis: $p(E \mid h_j) \approx \Pi_i\, p(e_i \mid h_j)$. Often works well even when the assumption is not justified.
- PROSPECTOR used Bayes for mineral exploration (found commercially significant deposits).

Worked example: car dealers

$p(d_1) = 0.2$, $p(d_2) = 0.4$, $p(d_3) = 0.4$. $p(a_1 \mid d_1) = 0.2$, $p(a_1 \mid d_2) = 0.4$, $p(a_1 \mid d_3) = 0.3$. You bought a_1. What is $p(d_2 \mid a_1)$?

$$p(d_2 \mid a_1) = (0.4)(0.4) \,/\, [(0.2)(0.2) + (0.4)(0.4) + (0.4)(0.3)] = 0.16 \,/\, 0.32 = \mathbf{0.5}$$

Problems of full Bayesian systems

- All probabilities relating evidence to hypotheses, **and between pieces of evidence**, must be known or estimated.
- Combinations of symptoms: about $m \times n^2 + n^2 + m$ numbers. With 200 diseases and 2000 symptoms that is **over 800,000,000**; even with only 10% dependent pairs, about 80,000,000.
- Negative evidence needed: $p(\neg e) = 1 - p(e)$. Note $p(e \mid h) \neq p(h \mid e)$.
- Tables must be rebuilt when new relationships are discovered. Hence Bayesian belief networks (Ch. 9) and d-separation.

5.4 Applications

Probabilistic finite state machine / acceptor

- A finite state machine whose **next-state function is a probability distribution** over all possible next states. It is an **acceptor** when one or more states are start states and one or more are accept states.

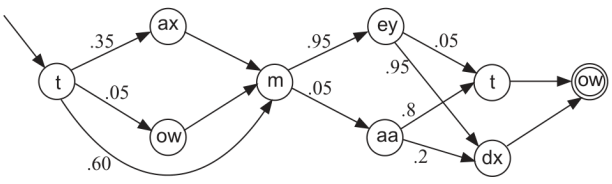


Figure 5.3 A probabilistic finite state acceptor for the pronunciation of tomato , adapted from Jurafsky and Martin (2009).

- **“Tomato” pronunciations:** each path from start to accept is a pronunciation; arc labels are the probabilities speakers take that transition (60% go directly from t to m).
- **Decoding the phone [ni] after “I”:** raw word frequency would choose “the” (p = .046). Using p(word | [ni]) ∝ p([ni] | word) p(word) ranks **new** first (0.00036), then neat, need, knee, and “the” = 0.
- But “I new” is unlikely. Use **n-gram** analysis (bigrams = word pairs, trigrams = triples): “I need” is far more frequent than “I new”.
- Corpora: Brown (1 million words, written) and Switchboard (1.4 million words, telephone speech).

Word	p([ni] | word)	p(word)	Product
new	0.36	0.001	0.00036
neat	0.52	0.00013	0.000068
need	0.11	0.00056	0.000062
knee	1.0	0.000024	0.000024
the	0.0	0.046	0.0

Traffic: explaining away

Road construction is correlated with orange barrels and bad traffic; accident with flashing lights and bad traffic. Seeing orange barrels makes construction the best explanation and accident is **explained away** (less likely, not impossible).

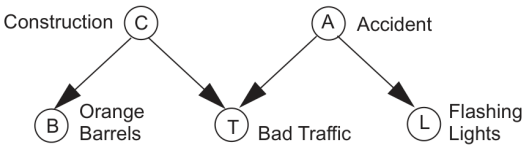


Figure 5.4 The Bayesian representation of the traffic problem with potential explanations.

C	T	p
t	t	.3
t	f	.2
f	t	.1
f	f	.4

$$p(C \mid T) = p(C=t, T=t) / [p(C=t, T=t) + p(C=f, T=t)] = .3 / (.3 + .1) = \mathbf{.75} \text{ (prior } p(C) = .5)$$

Complexity: the chain rule for five variables, p(C,A,B,T,L) = p(C) p(A|C) p(B|C,A) p(T|C,A,B) p(L|C,A,B,T), needs a table of size $2^5 = 32$; with 30 variables, about a billion entries.

Exam focus

- Counting: permutations, combinations, inclusion–exclusion (poker hands, committees).
- Compute probabilities from sample spaces and a joint table; marginalise.
- Test independence and conditional independence; use the chain rule.
- Apply Bayes’ theorem (dealers, medical test, defective products, three prisoners).
- Explain naive Bayes, MAP, parametric vs non-parametric, why full Bayes is intractable.
- Describe probabilistic FSMs and n-grams.

Chapter 6 — Building Control Algorithms for State Space Search

Part II so far: a solution is a **path in a graph**; search tries paths systematically; backtracking recovers from failure; **open** and **closed** lists record states (open as **stack** for depth-first, **queue** for breadth-first, **priority queue** for best-first). Chapter 6 adds recursion, production systems and blackboards.

6.1 Recursion-based search

A recursive procedure has (1) a **recursive step** (calls itself) and (2) a **terminating condition**. Recursion suits lists, trees and graphs. State space search is inherently recursive: move to a child and recur; if it fails try the next sibling.

```
function depthsearch(current_state);          % closed is global; open is implicit in the
  if current_state is a goal then return SUCCESS;     % activation records of the recursion
  add current_state to closed;
  while current_state has unexamined children
      child := next unexamined child;
      if child not member of closed then
          if depthsearch(child) = SUCCESS then return SUCCESS
  return FAIL
```

- Each call has an **activation record** (parameters, locals, execution state). The sequence of records on the call stack is the current path; failure returns to the parent, which tries the next sibling (**backtracking**).
- Without an explicit open list, **breadth-first and best-first are no longer natural extensions** of this version (they need open as a queue or priority queue).

Pattern-directed search (the basis of Prolog)

Goal-directed search over predicate calculus: the query is the goal, **unification** selects rules whose conclusion matches, and **modus ponens** generates transitions. The rule premise becomes a **subgoal**; search ends when a subgoal matches a fact. It **separates problem knowledge from control**.

```
function pattern_search(current_goal);
  if current_goal is a member of closed then return FAIL          % loop test
  else add current_goal to closed;
  while unifying facts or rules remain do
    case
      current_goal unifies with a fact:   return unifying substitutions;
      current_goal is negated (¬p):      call pattern_search on p;
                                          if it returns FAIL then return {} else FAIL;
      current_goal is a conjunction:      call pattern_search on each conjunct, applying
                                          substitutions to the other conjuncts;
                                          return composition of unifications (or FAIL);
      current_goal is a disjunction:      try each disjunct until one succeeds;
      current_goal unifies with rule conclusion (p in p ← q):
                                          apply substitutions to premise q; recurse;
                                          return composition of substitutions
  return FAIL
```

- Logic is **declarative**; a control regime must impose an order: try rules and facts in the order they appear in the knowledge base. The programmer controls search by ordering rules.
- Conjunctive goals must be solved with **consistent bindings**: p(X) ∧ q(X) needs the same X in both.
- **Negation as failure:** ¬p succeeds (with no bindings) only if p fails.

6.2 Production systems

Production system

- A model of computation for pattern-directed control of problem solving, consisting of: (1) a set of **production rules** (condition–action pairs, each a chunk of knowledge); (2) **working memory** (description of the current state); (3) the **recognise–act cycle**.

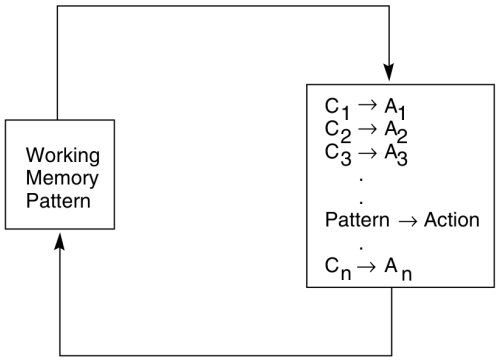


Figure 6.1 A production system. Control loops until working memory pattern no longer matches the conditions of any productions.

- **Cycle:** match working memory against rule conditions → **conflict set** (the enabled rules) → **conflict resolution** picks one → **fire** it (its action changes working memory) → repeat. Stops when no rule matches.
- The pure model has no recovery from dead ends; practical systems add **backtracking**.
- Origin: Post (1943) rewrite rules, equivalent in power to a Turing machine. Newell and Simon used it to model human problem solving: rules = long-term memory skills (unchanged by execution); working memory = short-term memory.
- Languages and shells: **OPS5** (XCON), **CLIPS** (NASA, C), **JESS** (Sandia, Java).

Example: sorting a string

Rule	Production
1	ba → ab
2	ca → ac
3	cb → bc

Iteration	Working memory	Conflict set	Rule fired
0	cbaca	1, 2, 3	—
1	cabca	2	2
2	acbca	2, 3	2
3	acbac	1, 3	1
4	acabc	2	2
5	aacbc	1	1
6	aabcc	—	Halt

Example: 8-puzzle and knight’s tour

- **8-puzzle productions:** goal state in working memory → halt; blank not on left edge → move left; not on top edge → up; not on right edge → right; not on bottom edge → down. Control: try rules in order, no loops, stop at goal; use a depth bound (5).
- **Knight’s tour (3 × 3 board, squares 1–9):** facts such as move(1,8), move(1,6). Recursive control: ∀X `path(X,X)` and ∀X,Y `[path(X,Y) ← ∃Z [move(X,Z) ∧ path(Z,Y)]]`. Trace for path(1,2): 1→8→3→4→9→2.
- **Loops** are hard to spot in production systems (rule order not fixed): move(2,9) then move(9,2). Fix: record visited squares **in working memory**, not in a global closed list. `assert(X)` adds X to working memory (an action, always succeeds); marker `been(Z)`:

```
∀X path(X,X)
∀X,Y [path(X,Y) ← ∃Z [move(X,Z) ∧ ¬(been(Z)) ∧ assert(been(Z)) ∧ path(Z,Y)]]
```

- **Full 8 × 8 tour:** replace move facts by eight rules, e.g. current row < 6 ∧ current column < 7 → new row = row + 2, new column = column + 1.
- **Financial advisor:** logical implications = productions; case data in working memory; fire until no top-level conclusions remain. JESS and CLIPS are production systems with a user interface, uncertainty handling and tracing added.

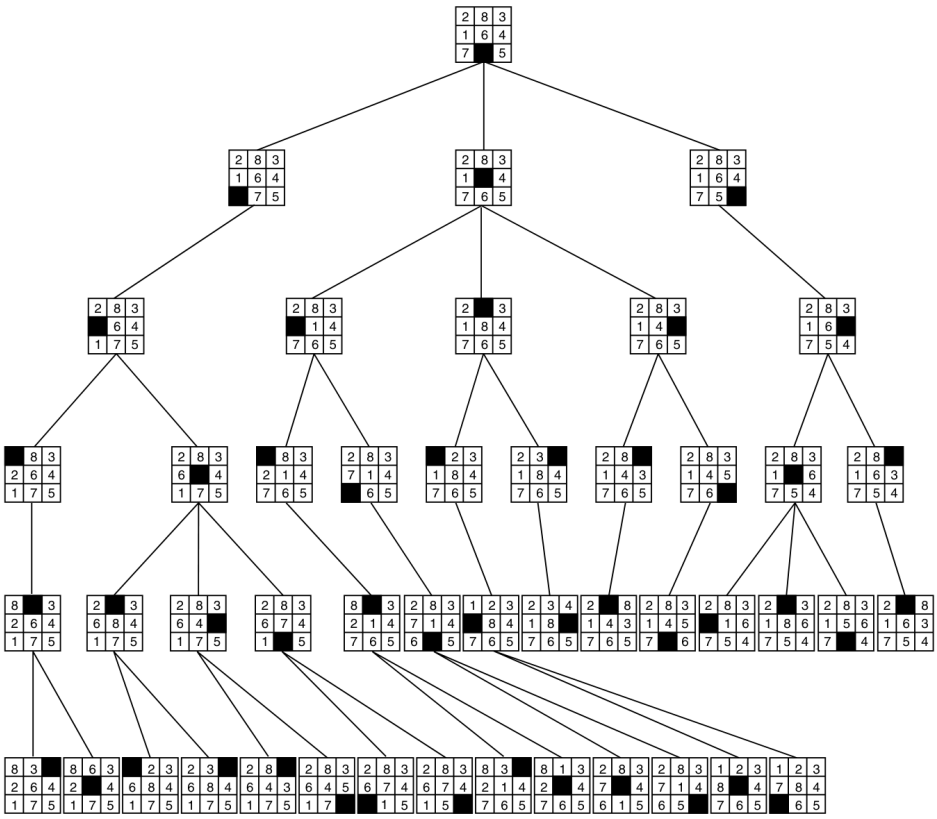


Figure 6.4 The 8-puzzle searched by a production system with loop detection and depth bound 5, from Nilsson (1971).

Control of search in production systems

Production systems can run **data-driven** or **goal-driven** on the same rules. Rules: 1 p∧q→goal; 2 r∧s→p; 3 w∧r→q; 4 t∧u→q; 5 v→s; 6 start→v∧r∧q.

Production set:

1. p ∧ q → goal
2. r ∧ s → p
3. w ∧ r → q
4. t ∧ u → q
5. v → s
6. start → v ∧ r ∧ q

Trace of execution:

Iteration #	Working memory	Conflict set	Rule fired
0	start	6	6
1	start, v, r, q	6, 5	5
2	start, v, r, q, s	6, 5, 2	2
3	start, v, r, q, s, p	6, 5, 2, 1	1
4	start, v, r, q, s, p, goal	6, 5, 2, 1	halt

Space searched by execution:



Figure 6.9 Data-driven search in a production system.

Production set:

1. p ∧ q → goal
2. r ∧ s → p
3. w ∧ r → p
4. t ∧ u → q
5. v → s
6. start → v ∧ r ∧ q

Trace of execution:

Iteration #	Working memory	Conflict set	Rule fired
0	goal	1	1
1	goal, p, q	1, 2, 3, 4	2
2	goal, p, q, r, s	1, 2, 3, 4, 5	3
3	goal, p, q, r, s, w	1, 2, 3, 4, 5	4
4	goal, p, q, r, s, w, t, u	1, 2, 3, 4, 5	5
5	goal, p, q, r, s, w, t, u, v	1, 2, 3, 4, 5, 6	6
6	goal, p, q, r, s, w, t, u, v, start	1, 2, 3, 4, 5, 6	halt

Space searched by execution:

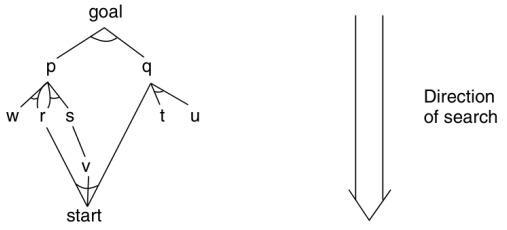


Figure 6.10 Goal-driven search in a production system.

	Data-driven	Goal-driven
Working memory starts with	Facts (start)	The goal
Matches against	Rule **conditions**	Rule **actions (conclusions)**
Firing	Adds the conclusion to working memory (modus ponens)	Adds the conditions as subgoals
Ends when	Goal derived	Subgoals reduce to known facts or user answers
Rules fired in the example	6, 5, 2, 1	1, 2, 3, 4, 5, 6

- **Combined strategies:** forward until the number of states gets large, then switch to goal-directed. With heuristic search the two frontiers may **miss** each other (Fig 6.11); with constant branching and exhaustive search the combination can cut the space drastically (Fig 6.12).
- **Rule structure and order:** a production system imposes **procedural semantics** on a declarative language. ∀X (foo(X) ∧ goo(X) → moo(X)) and ∀X (foo(X) → moo(X) ∨ ¬goo(X)) are logically equivalent but do not behave the same. Premise order encodes expert heuristics (“if the engine will not turn over and the lights do not come on, check the battery”).

Conflict resolution strategies (first three are from OPS5)

Strategy	Rule
Refraction	A rule that has fired may not fire again until the working memory elements matching it are modified (discourages looping)
Recency	Prefer rules matching the most recently added working-memory patterns (focuses on one line of reasoning)
Specificity	Prefer the more specific rule (more conditions, matches fewer patterns) over a general one
First match	Simplest regime: choose the first matching rule (or the first that leads to an unvisited state)

Advantages of production systems for AI

Advantage	Why
Separation of knowledge and control	Edit rules without changing control code, and vice versa
Natural mapping onto state space search	Working-memory states = nodes; rules = transitions; conflict resolution = branch choice
Modularity of rules	No syntactic interaction; rules affect each other only via working memory; incremental development
Pattern-directed control	Rules may fire in any order; the current state decides the path
Opportunities for heuristic control	Rule structure, rule order and conflict resolution
Tracing and explanation	Each rule is a meaningful chunk; chain of rules = line of reasoning
Language independence	Any representation supporting pattern matching
Plausible model of human problem solving	Newell and Simon; cognitive science

6.3 The blackboard architecture

Blackboards extend production systems by organising production memory into **modules (knowledge sources, KS)** that cooperate through a **global blackboard**.

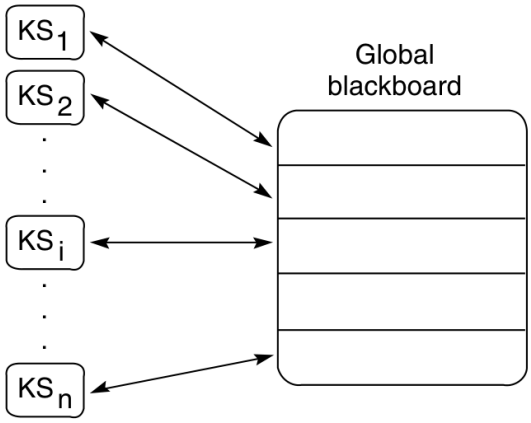


Figure 6.13 Blackboard architecture.

- Each KS is **independent** (own process), **asynchronous** and **data-driven**: it starts when it finds suitable input on the blackboard, processes it, and posts its results.
- Used where several different kinds of knowledge or processes must be coordinated: speech understanding, **sensor fusion** (tracking aircraft across radar sites).
- **HEARSAY-II** (speech understanding, front end to a library database): dimensions = time and level of analysis. KS levels: waveform → phonemes → syllables → words (two KSs) → word sequences → phrases. Competing hypotheses at each level; a KS can ask a lower one to try again. A **scheduler** KS uses a priority queue; when no KS is active it shuts down. Worked well with about 1,000 words.
- **HEARSAY-III:** generalisation (an expert system shell); drops the time dimension; splits the scheduler off as a separate **second blackboard** (controller).

Exam focus

- Write or trace recursive depth-first search; explain what recursion replaces (the open list).
- Explain pattern_search: unification, subgoals, conjunction, negation.
- Define the production system and its recognise–act cycle; trace a run (sorting, knight's tour, data- vs goal-driven).
- Explain loop detection with been(Z) and assert; conflict resolution: refraction, recency, specificity.
- List production system advantages; contrast with the blackboard and describe HEARSAY-II.

Chapter 7 — Knowledge Representation

Three themes of AI representation (Part III introduction): **weak method** problem solving, **strong method** problem solving, and **distributed / embodied / agent-based** approaches.

7.1 Three approaches to AI representation

	Weak method	Strong method	Agent-based / embodied
Era	1950s–60s	1970s–80s	Recent (robotics, internet)
Idea	General heuristics on syntactic form of states, domain independent	Large amounts of explicit domain knowledge	No centralised knowledge base or general inference; situated, autonomous, flexible agents
Examples	Logic Theorist, **GPS** (means–ends analysis)	Expert systems; rule such as "engine does not turn over and lights do not come on → battery or cables (.8)"	Brooks' subsumption, Veloso's robot soccer, internet agents
Weakness	No single heuristic works across all domains	Acquiring and organising the knowledge	Coordination, scale

- **Means–ends analysis (GPS):** compare the current state with the goal syntactically and choose an operator that reduces the difference (e.g. use de Morgan to remove ∧ if the goal has none).
- **Knowledge representation hypothesis (Brian Smith 1985):** any mechanically embodied intelligent process comprises structural ingredients that (a) an external observer takes to represent a **propositional account** of the knowledge, and (b) play a **formal role** in causing the behaviour that manifests it.
- **Representation as mapping:** the objects and relations of the domain map to the computational objects and relations of the program. Distinguish the **scheme** (data structure) from the **medium** (programming language), like data structures vs languages.
- General organisational principles to support: class hierarchies, definitions, exceptions, defaults, time, causality, uncertainty.

7.2 Associationist theories and semantic networks

Why not just logic?

Logical → means only a relation between truth values. "All cardinals are red", ∀X (cardinal(X) → red(X)), is logically equivalent to ∀X (¬red(X) → ¬cardinal(X)), so a white sheet of paper would count as **evidence that cardinals are red**, which is silly: the English sentence implies a (causal) correlation between class membership and properties.

Associationism (empiricist): the meaning of an object is its network of associations with other objects (snow → cold, white, slippery, ice).

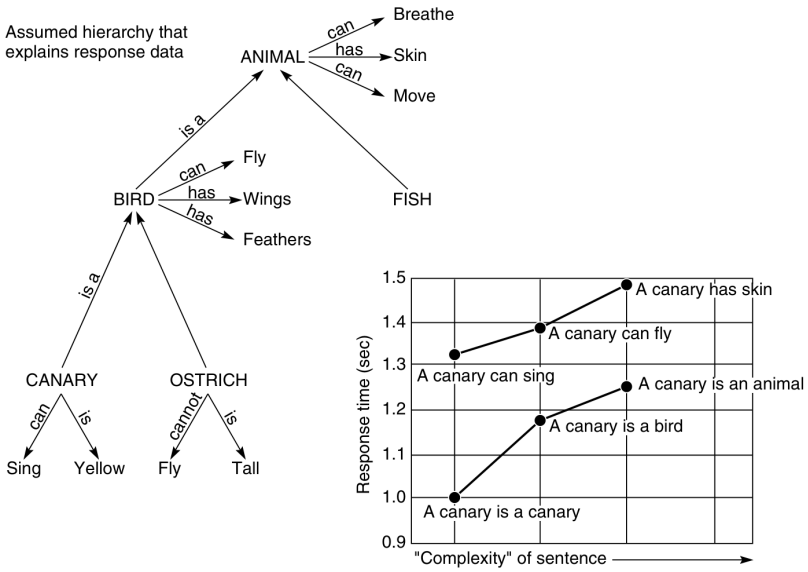


Figure 7.1 Semantic network developed by Collins and Quillian in their research on human information storage and response times (Harmon and King 1985).

- **Collins and Quillian (1969):** humans store information hierarchically at the most abstract level. Reaction times: “canary can sing” is faster than “canary can fly” (stored at *bird*), which is faster than “canary can breathe” (stored at *animal*). Exceptions (ostrich cannot fly) are stored at the most specific level. Formalised as **inheritance**: store properties at the highest level; reduces size and keeps updates consistent.
- **Semantic network:** a graph whose nodes are concepts or facts and whose labelled arcs are relations; supports inheritance. Roots: Porphyry’s type trees, Peirce’s existential graphs, Selz.
- **Quillian’s program (1967):** word concepts defined in planes (e.g. three senses of “plant”). Given two words it searches outward **breadth-first** for an **intersection node**. Uses: build text meaning from intersections, **disambiguate** words by shortest intersection path, answer flexible queries. Limited by the generality of its links.
- A graph notation alone has little advantage over logic. Power comes from **standard link types and inference rules** (inheritance); e.g. SNePS (Shapiro) was a first-order theorem prover.

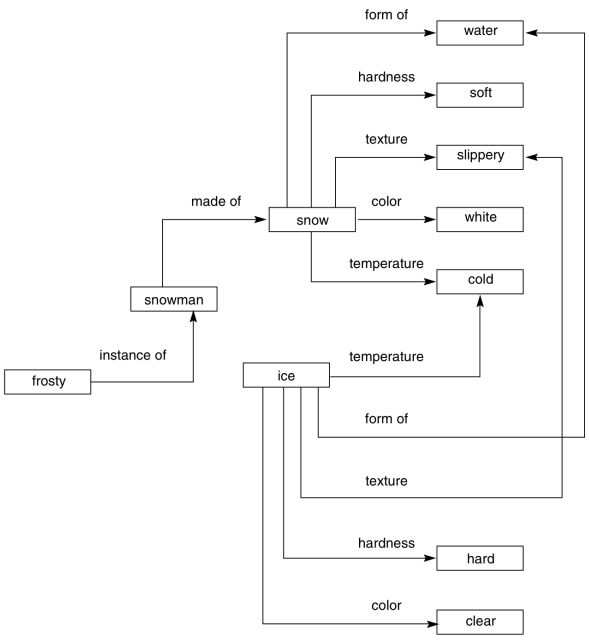


Figure 7.2 Network representation of properties of snow and ice.

Case frames and conceptual dependency

- **Case frame** (Simmons, from Fillmore): a verb node with case links (agent, object, instrument, location, time). “Sarah fixed the chair with glue”: agent = Sarah, object = chair, instrument = glue.
- **Conceptual dependency** (Schank): four primitive conceptualisations: **ACTs** (actions), **PPs** (objects, picture producers), **AAs** (modifiers of actions), **PAs** (modifiers of objects).
- 12 primitive ACTs (all actions reduce to these):

ACT	Meaning	ACT	Meaning
ATRANS	transfer a relationship (give)	MTRANS	transfer mental information (tell)
PTRANS	transfer physical location (go)	MBUILD	mentally make new information (decide)
PROPEL	apply physical force (push)	CONC	conceptualise, think about an idea
MOVE	move a body part by its owner (kick)	SPEAK	produce sound (say)
GRASP	grab an object (grasp)	ATTEND	focus a sense organ (listen)
INGEST	take in by an animal (eat)	EXPEL	expel from the body (cry)

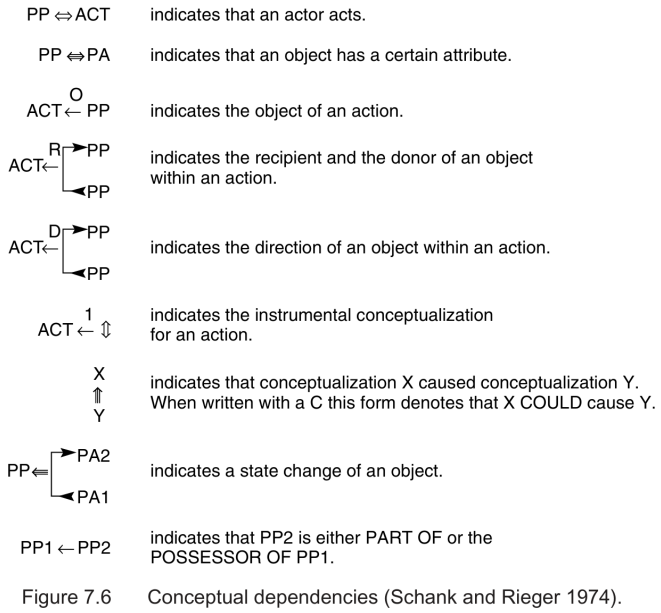


Figure 7.6 Conceptual dependencies (Schank and Rieger 1974).

- Tense and mode attachments: p past, f future, t transition, k continuing, ts start transition, tf finish transition, ? interrogative, c conditional, / negative, nil present, delta? timeless. “John ate the egg” = John ⇔ INGEST (past), object egg.
- **Benefit:** a **canonical form**: sentences with the same meaning get identical graphs, so equivalence is a simple graph match. **Criticisms:** reduction to canonical form is not reliably computable (provably uncomputable for monoids); no evidence humans do it; primitives too low-level and miss subtleties (“tall” vs fuzzy logic).
- Extensions: scripts and MOPs; MOPs underpin case-based reasoning.

7.3 Scripts and frames

Script (Schank and Abelson 1977)

- A structured representation of a **stereotyped sequence of events** in a context, built from conceptual dependency structures. Used to fill in missing information and default assumptions in natural language understanding.

Component	Restaurant example
Entry conditions	Restaurant open, customer hungry and has money
Results	Customer full and poorer; owner richer
Props	Tables, menus, waiter (support reasonable defaults)
Roles	Waiter takes orders and presents the bill; customer orders, eats, pays
Scenes	Entering, ordering, eating, exiting

- The program parses a story to conceptual dependency, matches entry conditions, binds people to roles and props to props, then answers questions (Did John eat dinner? Who is the “she” who ordered?).
- **Script match problem:** choosing the right script is “deep”, no algorithm guarantees it (“John visited his favourite restaurant on the way to the concert. He was pleased by the bill because he liked Mozart”: restaurant check or playbill?).
- **Between-the-lines problem:** unexpected events break a script (plaster falls on Melissa’s date). Scripts are inflexible.
- **MOPs (memory organisation packets):** smaller components combined dynamically; support case-based reasoning.
- Charniak: understanding “Mary was given two kites for her birthday so she took one back to the store” needs large background knowledge.

Frame (Minsky 1975)

- A remembered framework (static data structure) for a **stereotyped situation** with **named slots** and values, adapted to fit reality by changing details (hotel room, chair, hotel phone).

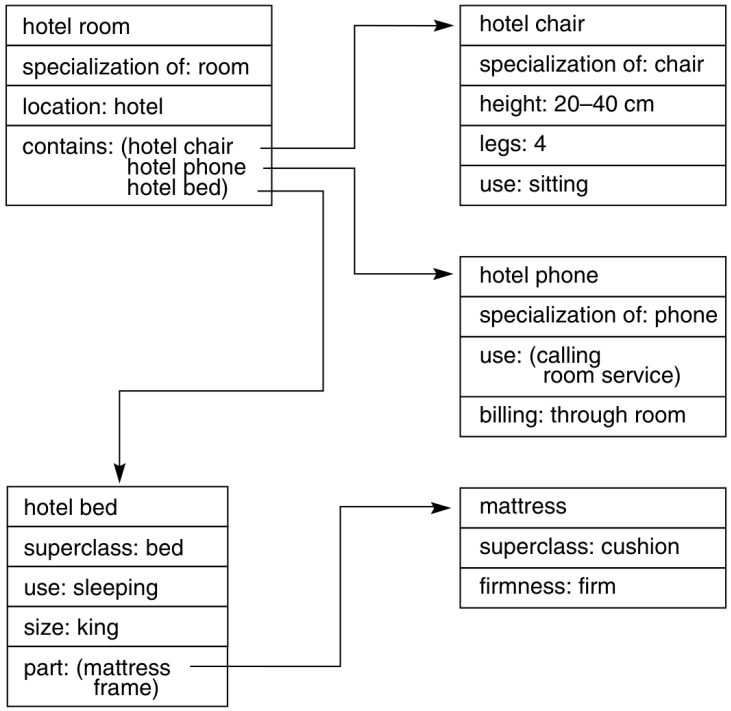


Figure 7.12 Part of a frame description of a hotel room. Specialization indicates a pointer to a superclass.

Slot content	Example
Frame identification	hotel chair
Relationship to other frames	hotel phone is a specialisation of phone
Descriptors of requirements	chair seat 20–40 cm from floor, back above 60 cm
Procedural information	Attached procedures (**procedural attachment**); **demons** run as side effects, e.g. consistency checks when a value changes
Defaults	Chairs have four legs; hotel beds are made by staff
New instance information	Colour of the bedspread (left unspecified until needed)

- Frames extend semantic networks: complex objects are **single entities**, organised hierarchically; support **class inheritance** and **defaults** (default reasoning is nonmonotonic).
- Minsky’s vision example: different views of a cube are recognised as one object by inferring hidden faces as defaults (Fig 7.13). Frame ideas led to object-oriented programming (Smalltalk, C++, Java).

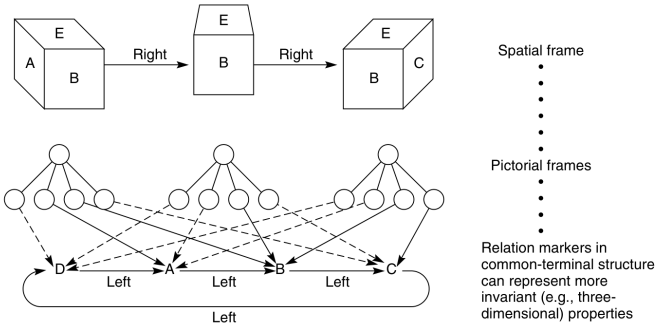


Figure 7.13 Spatial frame for viewing a cube (Minsky 1975).

7.4 Conceptual graphs (Sowa)

Conceptual graph

- A **finite, connected, bipartite** graph. Nodes are **concepts** (boxes) or **conceptual relations** (ellipses); arcs join concepts only to relations and vice versa. Each graph represents **one proposition**. A relation of arity n has n arcs.

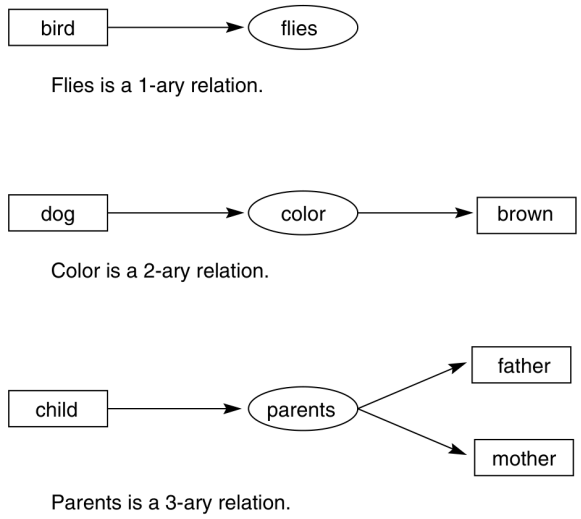


Figure 7.14 Conceptual relations of different arities.

- **Types, individuals, names:** label = type : referent (dog : emma). A **marker** #n identifies a unique individual; the **generic marker** * (or omission) denotes an unspecified individual; a named variable *X marks the same unspecified individual in two places.
- Separating an individual from its name allows: “John is a common name”, one individual with several names, several individuals with the same name.
- **Type hierarchy:** partial order < on types (t < s means t is a subtype of s). It forms a **lattice**: every pair has a minimal common supertype and maximal common subtype. Universal type **T** (supertype of all) and absurd type ⊥ (subtype of all).

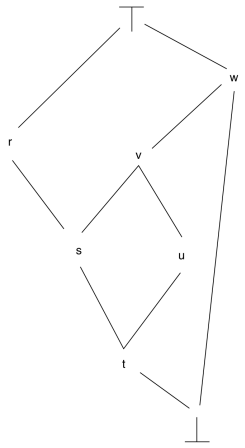


Figure 7.21 A type lattice illustrating subtypes, supertypes, the universal type, and the absurd type. Arcs represent the relationship.

Canonical formation rules

Rule	Effect
Copy	Make an exact copy of a graph
Restrict	Replace a generic marker by an individual marker, or a type label by a **subtype** (specialisation)
Join	Combine two graphs on an identical concept node (specialisation)
Simplify	Delete a duplicate relation (and its arcs)

- Join and restrict implement **inheritance** and plausible assumptions (“Mary and Tom went out for pizza”: join with pizza and restaurant graphs).
- **Not truth-preserving** (Emma might be a cat) but **meaning-preserving**: “Einstein plays centre for the Lakers” is false but meaningful; “conceptual graphs are yellow flying popsicles” is meaningless.
- **Propositional nodes:** a box containing a graph, used for belief and modal concepts (Tom believes Jane likes pizza; experiencer relation).
- **Logic:** negation with **neg** (“there are no pink dogs”), disjunction with **or**; generic concepts are existentially quantified. Equal in power to predicate calculus.
- **Translation to predicate calculus:** unique variable per generic concept, constant per individual, unary predicate per concept (its type), n-ary predicate per relation, conjunction of all, existentially quantified. dog:emma with colour brown becomes $\exists X1$ (dog(emma) $\wedge$ color(emma, X1) $\wedge$ brown(X1)).

7.5 Alternative representations and ontologies

Brooks’ subsumption architecture

- Intelligent behaviour comes from the interaction between a well-designed system and its **environment**, not from disembodied theorem provers. “Use the world as its own model.”
- A **layered** set of task handlers. Each layer is a fixed-topology network of simple **finite state machines** (few states, a register or two, a timer) that run **asynchronously**, passing fixed-length messages over wires. No central control, no global data, no dynamic links. Rules are condition → action. Higher layers **suppress or inhibit** lower ones.
- Three layers in the first robot (twelve sonar sensors): **AVOID** (halt, avoid obstacles), **WANDER** (random heading about every ten seconds, combined with avoidance), **EXPLORE** (plans a path to distant places, corrects for divergences). No centralised symbolic reasoning and no search through next states. Later robots collect drink cans.
- **Open questions:** (1) sufficiency of local information (myopia); (2) how can global coherence arise with no model of the environment? (3) how can a purely reactive component learn? (4) scale: what design principles extend it to complex behaviour? And: **what is “emergence”?** Without design instruction it merely names what we cannot yet explain.

Copycat (Mitchell and Hofstadter)

- A hybrid architecture for **analogy** (abc is to abd as ijk is to ?). Unlike Brooks it keeps a global state, and unlike static schemes its representation **evolves**.
- **Workspace:** global structure (like a blackboard) where perceptual structures and bonds are built. **Slipnet:** dynamically deformable semantic network with node **activation levels**. **Coderack:** priority-based probabilistic queue of **codelets** (small pieces of code exploring the problem). **Temperature:** measure of perceptual organisation; high temperature means more randomness in decisions, low temperature means a confident answer emerging.
- Perception is both **bottom-up** and **top-down**.

Ontologies and knowledge services

Complex applications integrate several representation schemes. Each scheme is an **ontology** (from the Greek for being); combining modules means **merging ontologies** into a **knowledge service** (e.g. a virtual meeting space). Languages: **OWL** family, SOFA. Uses: semantic web, image and text summarisation, agent skills.

7.6 Agent-based and distributed problem solving

- Two 1980s insights: the **DAI** (Distributed AI) community (first workshop at MIT, 1980; interest in coordination, not low-level parallelism) and Brooks' challenge to centralised knowledge. The environment can hold part of the solution, so one agent can check inventory while another checks credit, neither knowing the final decision.

> **Multi-agent system (Luger)**
> - A program with problem solvers situated in interactive environments, each capable of flexible, autonomous, yet socially organised action. Four criteria: **situated, autonomous, flexible, social**.

Criterion	Meaning	Example
Situated	Receives input from, and can change, its environment	ROBOCUP soccer player
Autonomous	Acts without direct intervention; controls own actions and state; may learn	Credit card check independent of the rest of a purchase
Flexible	**Responsive** (timely reaction) and **proactive** (plans, opportunistic, goal-directed)	Credit agent tries another agency
Social	Interacts with other agents: bidding, communication, supporting goals	Pass the ball to a teammate

- **Four characteristics:** each agent has incomplete information and limited capability; no global controller; decentralised knowledge and data; asynchronous reasoning. Advantages: distributed, concurrent solving; interactions = **cooperation, coordination, negotiation**.
- **Agents vs objects:** objects have encapsulated state, methods and message passing but rarely control their own behaviour; agents *request* services, are reactive, proactive and social, and often have their own thread of control. OO languages are still a good medium for agents.
- **Application areas:** manufacturing, automated control (spacecraft, particle accelerators, air traffic), telecommunications, transportation, information management (filtering, gathering), e-commerce (portfolio management, shopping), interactive games and theatre.
- **Challenges:** decompose, allocate and synthesise problems; communication languages and protocols; coherent action and avoiding harmful interaction; modelling other agents' plans; conflicting viewpoints; chaotic or oscillatory behaviour; resource allocation; hardware and software platforms.

7.7 Issues in representation (epilogue)

Issue	Point
Granularity of atomic symbols	If “car” is the smallest atom you cannot reason about wheels; finer atoms need larger structures for “car”. Semantic primitives avoid single-word units but cannot express push vs shove.
Exhaustiveness	All occurrences of a class or property correspond to an explicit element (a complete map). If exhaustive, the **frame problem** effectively disappears.
Plasticity	Easy to modify; keeps consistency (inheritance helps). Copycat and Brooks’ robot evolve their structures.
Homomorphism	One-to-one correspondence between world and representation objects and actions; more natural and intuitive.
Efficiency vs expressiveness	Unconstrained logic is highly expressive but pays heavily in efficiency.

Frame problem: representing the side effects of actions, i.e. what does *not* change (a robot loading boxes lowers the truck bed).

Exam focus

- Weak vs strong method problem solving vs agent-based (examples of each).
- Why logic is awkward for commonsense (cardinal example); associationism and Collins–Quillian.
- Semantic nets, case frames, conceptual dependency (ACTs, canonical form and its criticisms).
- Scripts (components, match and between-the-lines problems) and frames (slots, defaults, demons, inheritance).
- Conceptual graphs: type hierarchy as a lattice, restrict/join/copy/simplify, translation to predicate calculus.
- Subsumption architecture, Copycat (workspace, slipnet, coderack, temperature), agent criteria.

Chapter 8 — Strong Method Problem Solving

"The power resides in the knowledge." (Feigenbaum). Expert systems must be **knowledge-rich even if methods-poor**: performance comes primarily from the knowledge base and only secondarily from the inference method.

8.1 Expert system overview

Expert systems (1) support **inspection** of their reasoning (intermediate steps, answers to questions), (2) allow **easy modification** of the knowledge base, and (3) reason **heuristically** with imperfect knowledge. Explanations matter because experts will not accept advice they cannot justify; in a pure production system changing one rule has no global syntactic side effects.

Category (Waterman 1986)	Task
Interpretation	High-level conclusions from raw data
Prediction	Probable consequences of given situations
Diagnosis	Cause of malfunctions from observable symptoms
Design	Configuration meeting goals under constraints
Planning	Sequence of actions to achieve goals
Monitoring	Compare observed with expected behaviour
Instruction	Assist education in technical domains
Control	Govern the behaviour of a complex environment

Architecture

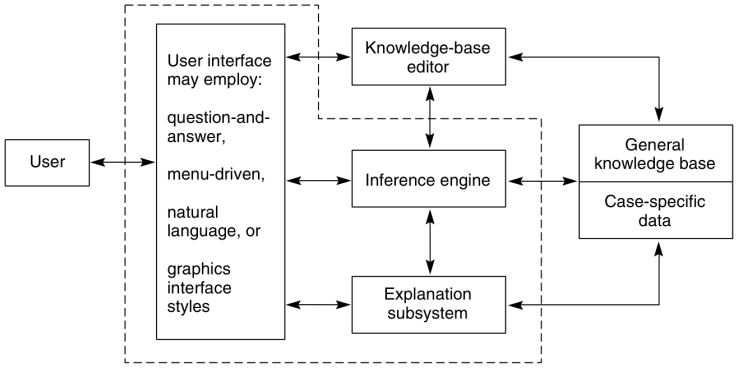


Figure 8.1 Architecture of a typical expert system for a particular problem domain.

Module	Role
User interface	Hides complexity (question-and-answer, menu, graphical)
Knowledge base	Domain knowledge, usually if…then rules; general knowledge plus case-specific data
Inference engine	Interpreter for the knowledge base; performs the recognise–act cycle
Case-specific data	Facts, partial conclusions, certainty measures, dead ends for the current case
Explanation subsystem	Answers **how** and **why** queries; justifications
Knowledge-base editor	Finds and fixes bugs, maintains rule syntax, consistency checks
Shell	All modules with an empty knowledge base and case data (CLIPS, JESS)

Why separate knowledge from the inference engine: knowledge in a natural form (if…then), builders focus on knowledge, changes without side effects, and the same control and interface software can be reused.

Is a problem suitable? (seven guidelines)

1. The need justifies the cost and effort.
2. Human expertise is not available everywhere it is needed.
3. The problem can be solved by **symbolic reasoning** (no dexterity or perception).

4. The domain is **well structured** and does not need commonsense reasoning.
5. It cannot be solved satisfactorily by traditional computing.
6. Cooperative, articulate experts exist.
7. The problem has the proper size and scope.

People and process

- **Knowledge engineer:** AI language and representation expert; selects tools, helps the expert articulate knowledge, builds the knowledge base. Often initially ignorant of the domain (a benefit: they notice skipped steps). **Domain expert:** supplies the knowledge, shortcuts and heuristics. **End user:** sets the major design constraints (interface, explanation level, environment).
- **Exploratory programming:** early prototype, test with the expert, correct and extend; the knowledge base is “grown”. The prototype may be thrown away. The program is never “finished”.
- **Conceptual model:** the knowledge engineer’s evolving conception of the domain, between human expertise and the formal program; documented (requirements, data dictionaries, recorded interviews).
- **Difficulties of acquisition:** skill is often unconscious; expertise is knowing how to cope, not a rational description (unicycle); knowledge is a model of the world shaped by convention; expertise changes.

8.2 Rule-based expert systems

The architecture is a **production system**: knowledge base = rules, working memory = case data, inference engine = recognise–act cycle (data-driven or goal-driven).

```
Rule 1: if the engine is getting gas and the engine will turn over then the problem is spark plugs.
Rule 2: if the engine does not turn over and the lights do not come on then the problem is battery or cables.
Rule 3: if the engine does not turn over and the lights do come on then the problem is the starter motor.
Rule 4: if there is gas in the fuel tank and there is gas in the carburetor then the engine is getting gas.
```

Goal-driven (backward) reasoning

- Put the goal “the problem is X” in working memory. Rules 1, 2, 3 match; choosing the lowest-numbered (rule 1) binds X = spark plugs and places its premises in working memory as subgoals. This chooses an **or** branch; the premises are **and** branches.
- Rule 4 matches “the engine is getting gas”. Subgoals that match no rule conclusion are put to the **user**. If the answers confirm all, the diagnosis is spark plugs; otherwise back up to rule 2.
- Backward search corresponds to **hypothesis testing** in human problem solving.

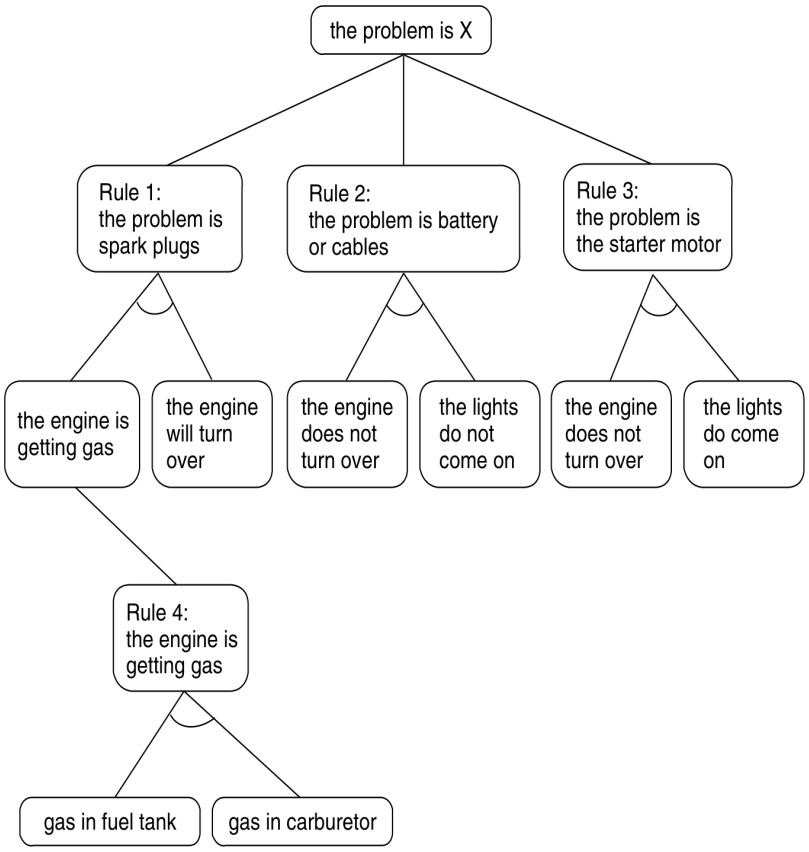


Figure 8.8 . The and/or graph searched in the car diagnosis example, with the conclusion of Rule 4 matching the first premise of Rule 1.

Explanation: why and how

Query	Meaning	Answer
why	Why do you ask for this information? (asked in reply to a system question)	The **current rule** being fired (premises established so far and the one needed)
how	How did you reach this conclusion?	The **trace of rules** and user answers that support it

These work because each rule is a meaningful chunk of reasoning. Rules should each be **one logical step**; otherwise explanations seem vague and illogical.

Data-driven (forward) reasoning

- Compare working memory with the conditions of each rule in order; if satisfied, add the conclusion to working memory; loop through the rules again. A fact is **askable** only if no rule concludes it.
- **Opportunistic search:** when a rule fires with new information, next consider rules that have that information as a premise.
- Compared with goal-driven: **less focused**, explanations are limited (rule-level why; a list of fired rules or working-memory contents for how).

Heuristics and control

- Control by **rule order and premise order**: put the premise most likely to fail or cheapest to test first (ask “will the engine turn over?” before the costly “is it getting gas?”). Heuristics may be wrong (a bad distributor).
- Staged problem solving: first premise of each rule names the stage (organise situation, collect data, analyse, report); last rule of a stage retracts one stage fact and asserts the next.
- **RETE** (Forgy) compiles rules into a network for fast matching without changing semantics.
- Summary: rules capture experience but are **brittle** at the edges of the domain, which model-based and case-based methods address.

8.3 Model-based, case-based and hybrid systems

Model-based reasoning (MBR)

First-generation expert systems used heuristic rules that link symptoms straight to diagnoses (MYCIN reflects results of infection, not causes) and fail outside their rules. A **model-based** reasoner uses a (usually qualitative) **software simulation of the structure and function** of the device. Early model-based systems were tutors (deKleer, Brown) for circuits.

- **Three components:** (1) description of each component (simulates its behaviour); (2) description of the internal structure (components and interconnections); (3) observations of actual behaviour (I/O measurements). Plus rules for known failure modes. A fault is a **discrepancy** between predicted and observed behaviour.
- Representation options: rules, relational tuples, constraints, objects (slots = state, methods = function).

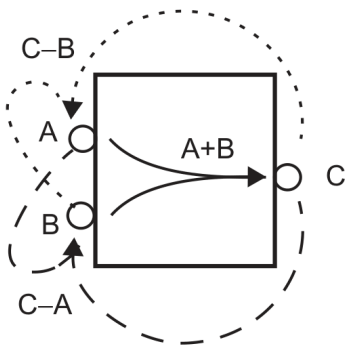


Figure 8.13 The behavior description of an adder, after Davis and Hamscher (1992).

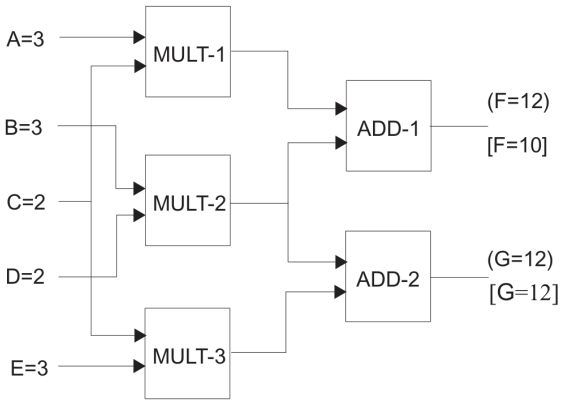


Figure 8.14 Taking advantage of direction of information flow, after Davis and Hamscher (1988).

Adder: if A and B known then C = A + B; if C and A known then B = C − A; if C and B known then A = C − B. In the circuit, F should be 12 but is 10 while G is correct. F depends on ADD-1, MULT-1 and MULT-2; reasoning back, the fault is in MULT-1 or ADD-1 (single-fault assumption).

Task	Meaning
Hypothesis generation	From a discrepancy, which components could have caused it?
Hypothesis testing	Which of those can actually explain the observed behaviour?
Hypothesis discrimination	What further information separates the surviving hypotheses?

- **Strengths:** robust, uses functional knowledge, transferable, causal explanations. **Weaknesses:** knowledge acquisition is demanding, programs are large and slow; the model is an idealisation. **Bridging faults** (unintended contacts) are hard because of an implicit **closed world assumption**: whatever is not in the model does not exist.

NASA example: Livingstone and Deep Space 1

- Autonomous spacecraft must reconfigure after failures with no pre-flight enumeration of contingencies. **Livingstone** is a model-based reactive self-configuring kernel using **propositional calculus** and a single model for plan tracking, confirming modes, reconfiguring, anomaly detection, diagnosis and recovery.
- Each component (valves, tanks, engines in Cassini's propulsion system) is a **transition system** with operating and failure modes. A **configuration manager** moves the craft to the lowest-cost configuration that achieves goals. **Pyro valves** fire once only (irreversible, high power); latch valves are reversible.
- **Mode estimation (ME)** infers the current configuration from the previous one plus observations (all transitions consistent with sensor values, most likely kept). **Mode reconfiguration (MR)** finds the least-cost commands that achieve the goal in the next state. Both are **reactive**; solved as combinatorial optimisation by conflict-directed best-first search. Component transitions are modelled as **synchronous**.

Case-based reasoning (CBR)

Uses an explicit database of past problems and solutions (law precedents, hardware diagnosis, medical case histories: CASEY, PROTOS). Knowledge acquisition is simple (store more cases); the system **learns** by saving new solutions with success or failure.

1. **Retrieve** appropriate cases (similar on salient features; indexed by features).

2. **Modify** the retrieved case to fit the current problem (analytic or heuristic adaptation).
3. **Apply** the transformed case (may need further iterations).
4. **Save** the solution with a record of success or failure, updating the index.

Kolodner's preference heuristic	Meaning
Goal-directed	Retrieve cases with the same goal as the current situation
Salient-feature	Prefer cases matching the most important (or most) features
Specificity	Prefer as exact a match as possible before general matches
Frequency	Check the most frequently matched cases first
Recency	Prefer the most recently used cases
Ease of adaptation	Use first the cases most easily adapted

- **Hard issues:** choosing **salient features** for indexing (e.g. weak signal and low power matter; the timestamp usually does not), defining **similarity**, **store/compute trade-off** (retrieval slows as the case base grows; keep prototypes, forget redundant or unused cases), and weak explanations ("it worked before"). Satellite example: repeated solar-panel reorientation fixed symptoms but a model-based reasoner found an overheating gyroscope.
- **Transformational analogy** (Carbonell): operators transform whole past solutions in a higher-level T-space (insert or delete steps, reorder, splice, change bindings).

Comparison and hybrids

Approach	Advantages	Disadvantages
Rule-based	Direct use of experiential knowledge; rules map to state space search; knowledge separated from control; good in limited domains; good explanations	Highly heuristic, no functional knowledge; brittle; degrades at domain edges; descriptive explanations only; task dependent
Case-based	Encodes history directly; shortcuts reasoning; avoids past errors; no deep domain analysis; indexing adds insight	No deeper knowledge (misapplication, weak explanation); store/compute trade-off; indexing and matching are hand-crafted
Model-based	Uses functional and structural knowledge; robust; knowledge transferable; causal explanations	No experiential knowledge; needs an explicit model; high complexity; exceptional situations (bridging faults)

- **Hybrids:** rule + case (check known cases first; record search results as cases); rule + model (functional explanations; fall back to first principles when rules fail; heuristic search in the model); model + case (richer explanations; cases guide model inference). Problems: when to switch methods, resolving differences, shared representations.

8.4 Planning

A planner finds a sequence of actions that lets a problem solver (e.g. a robot controller) reach a goal. Planning is **state space search** over world states. Traditional plans were produced before the robot acted.

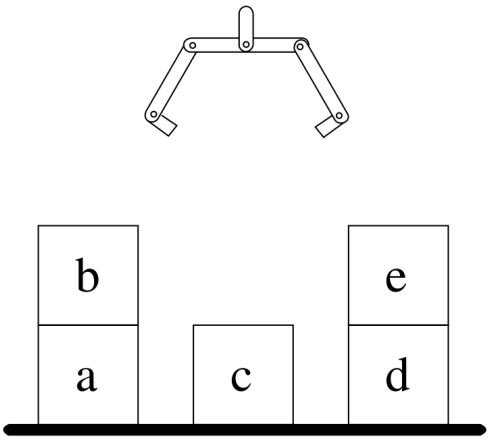


Figure 8.18 The blocks world.

- **Operators:** pickup(W), putdown(W), stack(U,V), unstack(U,V) (goto is implicit via recorded locations).
- **State predicates:** on(X,Y), clear(X), ontable(W), gripping(X), gripping( ), location(W,X,Y,Z).

- STATE 1: ontable(a), on(b,a), clear(b), ontable(c), on(e,d), clear(c), ontable(d), gripping( ), clear(e).
- STATE 2 (after unstack(e,d)): ontable(a), on(b,a), clear(b), ontable(c), clear(c), **clear(d)**, ontable(d), **gripping(e)**, clear(e).

Frame problem

- The problem of specifying what is true in a state and exactly **what an action changes** and what stays unchanged (McCarthy and Hayes 1969). **Frame axioms** (e.g. stack and unstack do not affect ontable(X)) can add exponentially to the search, and every new predicate (colour, size) needs new axioms.

STRIPS (Fikes and Nilsson)

Used to drive the SHAKEY robot. Each operator has **preconditions P**, an **add list A** and a **delete list D**. Everything not mentioned stays true, so no frame axioms are needed (precondition–add–delete is equivalent to inference rules plus frame axioms). The operator table is in Chapter 2 (blocks world).

- **Conflicting (incompatible) subgoals:** assuming subgoals are independent (linear) is wrong. Goal on(b,a) ∧ on(a,c) from STATE 1: on(b,a) is already true and must be undone before on(a,c).
- STRIPS saves successful plans as generalised **macro operators** in **triangle tables**. Macros prune search and avoid subgoal conflicts, but many macros add matching overhead.

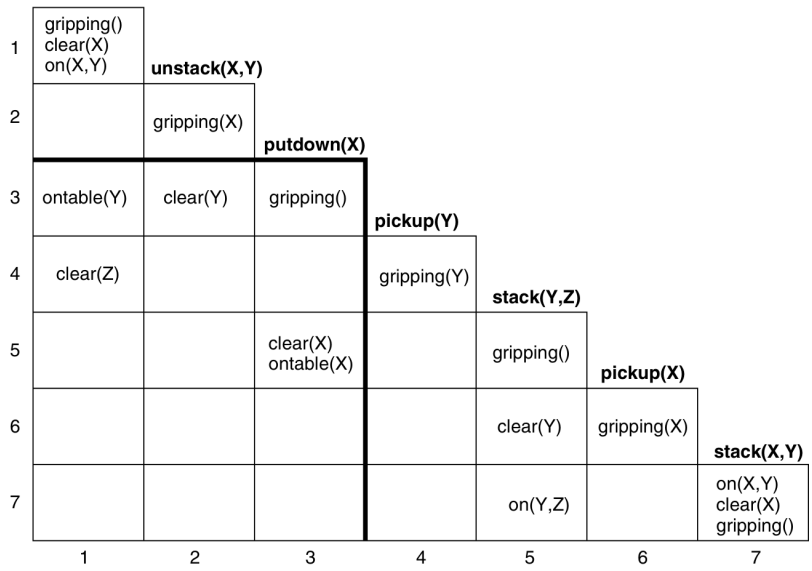


Figure 8.21 A triangle table, adapted from Nilsson (1971).

- **Triangle table:** actions on the diagonal; preconditions of each action in the row before it; postconditions (add/delete) in the column below it. The **leftmost column** gives the preconditions of the whole macro, the **bottom row** what it adds.
- **Kernel:** the nth kernel is the intersection of all rows below and including row n and all columns left of and including column n. The ith action may run only if all predicates in the ith kernel are true. To recover from accidents, execute the **highest-numbered action whose kernel is enabled** (back up or jump forward).
- Triangle tables keep only postconditions that are preconditions of later steps, so guaranteed correctness is weaker.

Teleo-reactive (TR) planning (Nilsson, Benson)

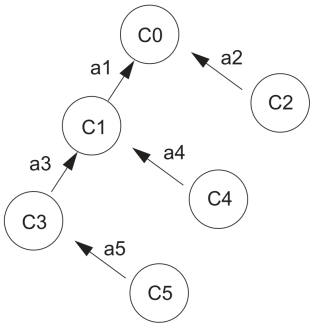


Figure 8.22 A simple TR tree showing condition action rules supporting a top-level goal, from Klein et al. (2000).

- A **TR tree** is a set of condition→action pairs $C_0 \rightarrow A_0$, $C_1 \rightarrow A_1$, ..., $C_n \rightarrow A_n$. C_0 is the top-level goal and A_0 the null action. Each cycle evaluates conditions **top to bottom** and performs the action of the first true one, repeating at a high frequency.
- Actions are **durative**: sustained while their preconditions hold and the goal is unmet; interrupted when an action nearer the goal becomes active. Execution is **adaptive** (falls back if effects are reversed) and **opportunistic** (jumps ahead if something good happens).
- Combines top-down hierarchical control with bottom-up agent-style sense–react cycles. Built by goal reduction from the top-level goal, regressing to the current state. If no condition is satisfied, replan. Used for distributed robots, flight simulation and charged-particle accelerator tuning (dynamic noisy systems, durative tuning actions, easily encoded by physicists).
- **NASA again:** Burton (follow-on to Livingstone) plans a sequence of control actions and removes the assumption that all interleavings of transitions are correct (TechSat 21).

Exam focus

- Why knowledge is power; characteristics and components of an expert system; shell.
- Trace goal-driven and data-driven rule runs; answer why and how queries.
- Compare rule-based, model-based and case-based reasoning; give hybrid designs.
- Model-based diagnosis: generation, testing, discrimination; bridging faults; Livingstone ME and MR.
- Frame problem, frame axioms vs add/delete lists, conflicting subgoals, triangle tables and kernels.
- Teleo-reactive trees vs STRIPS (durative actions, reactivity).
- Knowledge engineering roles and the development cycle.

Cheat sheet: key formulas, algorithms and comparisons

Search algorithms

Algorithm	open is a…	Shortest path?	Space	Notes
Backtrack	NSL + SL + DE	No	Linear	Depth-first with dead-end list
Depth-first	Stack (LIFO)	No	Linear in path length	Can follow infinite paths; use depth bound
Breadth-first	Queue (FIFO)	Yes	Exponential (B^n)	Level by level
Iterative deepening	DFS with bound 1, 2, 3…	Yes	Linear (B × n)	Time $O(B^n)$
Hill climbing	None (keeps best child)	No	Constant	Local maxima, no recovery
Best-first	Priority queue by heuristic	No	Exponential	Recovers via open
A (f = g + h)	Priority queue by f	If h ≤ h* (A*)	Exponential	Admissible when h ≤ h*
Minimax / alpha-beta	Depth-first to ply n	n/a	Linear	Alpha-beta prunes, same value

Formulas

Topic	Formula
Evaluation function	f(n) = g(n) + h(n); A*: h(n) ≤ h*(n)
Monotone heuristic	$h(n_i) - h(n_j) \le cost(n_i, n_j)$; h(goal) = 0
More informed	$h_1(n) \le h_2(n)$ for all n ⇒ h_2 more informed
States generated	$T = B + B^2 + \ldots + B^L = B(B^L - 1)/(B - 1)$
Search space sizes	Tic-tac-toe 9! = 362,880; TSP (N−1)!; chess about 10^{120}; checkers about 10^{40}
Permutations / combinations	nPr = n!/(n−r)!; nCr = n!/[(n−r)! r!]
Probability	p(E) = |E|/|S|; $p(\bar{E}) = 1 - p(E)$; p(A∪B) = p(A) + p(B) − p(A∩B)
Independence	p(A∩B) = p(A)p(B); conditional: p(A∩B | C) = p(A | C)p(B | C)
Conditional / Bayes	$p(d|s) = p(d \cap s)/p(s)$; $p(h_i|E) = p(E|h_i)p(h_i) / \Sigma_k\, p(E|h_k)p(h_k)$
Binomial / expectation	$nCr\ p^r(1-p)^{n-r}$; ex(E) = r·p(E) + c·(1−p(E))
Edit distance costs	insert 1, delete 1, replace 2 (match 0)
Euler path	Exists iff the number of odd-degree nodes is 0 or 2

Reasoning styles

	Data-driven (forward)	Goal-driven (backward)
Starts from	Facts	Goal or hypothesis
Good when	Data given, many goals, goal hard to state	Goal known, many rules fan out, data must be acquired
Production system match	Facts vs rule conditions	Goal vs rule conclusions

Knowledge representation at a glance

Scheme	Key idea
Predicate calculus	Sound, complete inference; declarative; undecidable in general
Semantic network	Graph of concepts; associations; inheritance
Conceptual dependency	12 primitive ACTs; canonical form
Script / MOP	Stereotyped event sequences; default assumptions
Frame	Slots, defaults, procedural attachment (demons), inheritance
Conceptual graph	Bipartite; type lattice; copy, restrict, join, simplify
Rules / production system	Condition→action; working memory; recognise–act cycle
Model-based, case-based	Simulate structure and function; reuse past solutions (retrieve, modify, apply, save)
Subsumption, Copycat, agents	No central model; layered FSMs; emergent behaviour; situated, autonomous, flexible, social